

(LAB)DIGITAL ASSIGNMENT - 4

PENETRATION TESTING AND VULNERABILITY ASSESSMENT

NAME – NORI KARTHIKEY
REG NO – 22BCI0052
SUBMITTED TO – DR. PRAKASH G.
COURSE CODE: (BCSE319P)

Exercise 3.1 Wireshark DNS

Question 1: Run nslookup to obtain the IP address of a Web server in Asia.

Ans

nslookup(Name Server Lookup) is a command-line tool used to query DNS servers. By default, it requests the '**A**' **Record** (an Address record) for a given hostname, which maps that name to an IPv4 address.

```
File Actions Edit View Help
(kali@kali) [~]
$ nslookup www.sony.jp
Server:      172.17.96.1
Address:     172.17.96.1#53

Non-authoritative answer:
www.sony.jp      canonical name = www.sony.jp.edgesuite.net.
www.sony.jp.edgesuite.net canonical name = a488.r.akamai.net.
Name:   a488.r.akamai.net
Address: 96.16.204.187
Name:   a488.r.akamai.net
Address: 96.16.204.188

(kali@kali)-[~]
$
```

Question 2: Run nslookup to determine the authoritative DNS servers for a university in Europe.

Ans. DNS stores different record types. To find the *authoritative* DNS servers (the official servers for a domain), we must query for the '**NS' Record** (Name Server record). We can do this using the -type=NS option in nslookup

```
(kali@kali)-[~]
$ nslookup -type=NS cam.ac.uk
Server:          172.17.96.1
Address:         172.17.96.1#53

Non-authoritative answer:
cam.ac.uk       nameserver = ns3.mythic-beasts.com.
cam.ac.uk       nameserver = dns0.cl.cam.ac.uk.
cam.ac.uk       nameserver = ns1.mythic-beasts.com.
cam.ac.uk       nameserver = dns0.eng.cam.ac.uk.
cam.ac.uk       nameserver = ns2.ic.ac.uk.
cam.ac.uk       nameserver = auth0.dns.cam.ac.uk.

Authoritative answers can be found from:
```

Question 3: Run nslookup so that one of the DNS servers obtained in Question 2 is queried for the mail servers for Yahoo! Mail.

Ans. nslookup allows you to specify *which* DNS server to query by adding it to the end of the command. To find mail servers, we query for the '**MX**' **Record** (Mail Exchange record), which tells email systems where to send mail for a domain.

```
(kali@kali)-[~]
$ nslookup -type=MX yahoo.com 172.17.96.1
Server:          172.17.96.1
Address:         172.17.96.1#53

Non authoritative answer:
yahoo.com       mail exchanger = 1 mta5.am0.yahoodns.net.
yahoo.com       mail exchanger = 2 mta6.am0.yahoodns.net.
yahoo.com       mail exchanger = 3 mta7.am0.yahoodns.net.

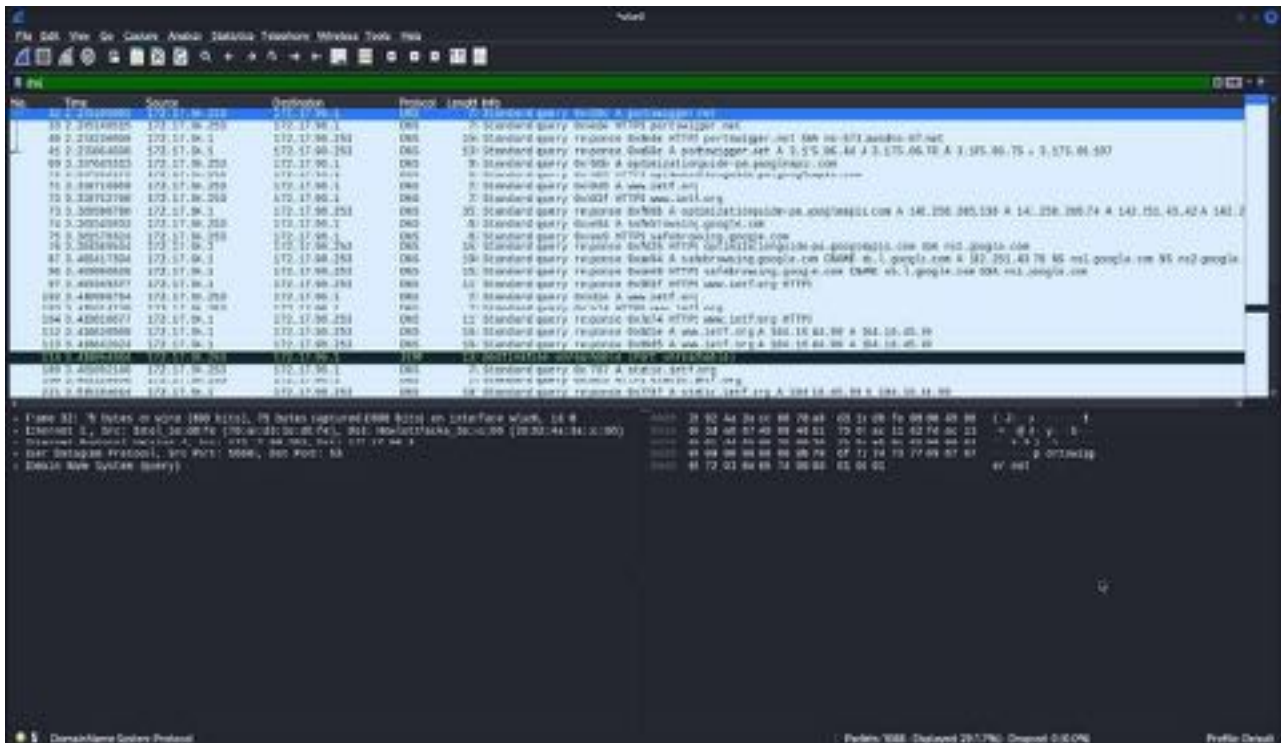
Authoritative answers can be found from:
```

Tracing DNS with Wireshark

Steps

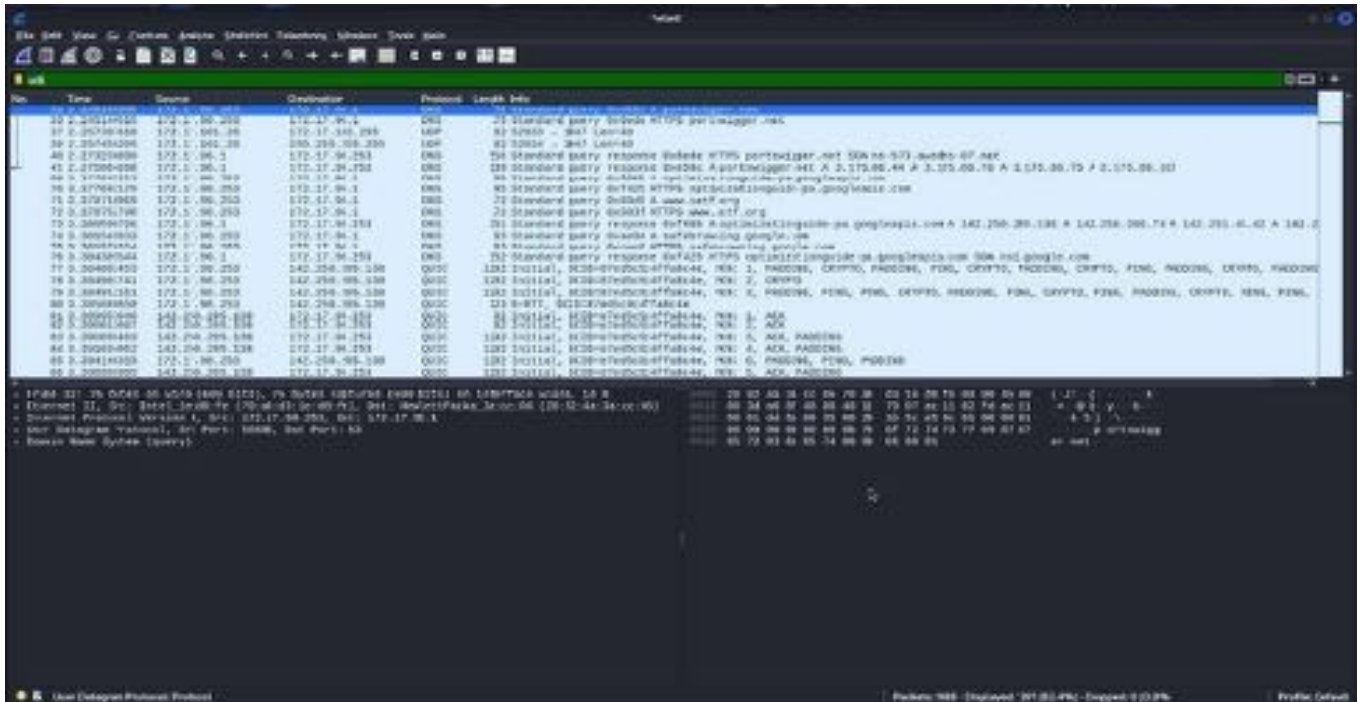
Flush DNS Cache (Kali): Open a terminal and run `sudo systemd-resolve --flush-caches`. This clears your computer's local DNS cache, forcing it to make new queries.

1. **Clear Browser Cache:** Open your web browser and clear its cache (in Firefox/Chrome, press Ctrl+Shift+Del).
2. **Start Wireshark:** Open a terminal and run `sudo wireshark` (running as sudo is the easiest way to ensure you can capture packets).
 - Select your active network interface (e.g., `eth0` or `wlan0`) and click the blue "Start" fin.
3. **Browse:** Go to `http://www.ietf.org` in your browser.
4. **Stop Capture:** Once the page loads, go back to Wireshark and click the red "Stop" square.
5. **Filter:** In the Wireshark display filter bar, type `dns` and press Enter.



Question 4: Locate the DNS query and response messages. Are they sent over UDP or TCP?

Ans DNS typically uses **UDP** (User Datagram Protocol) for standard queries. UDP is "connectionless" and very fast, making it ideal for a simple, small query-and-response. TCP is only used for larger, more complex DNS operations



Question 5: What is the destination port for the DNS query message? What is the source port of DNS response message?

Ans. The DNS protocol operates on the well-known standard **port 53**. Client queries are sent *to* destination port 53. The server then sends a response *from* source port 53.

```
Wireshark - Packet 32 - eth0

* Frame 32: 75 bytes on wire (600 bits), 75 bytes captured (600 bits) on interface wlan0, id 0
* Ethernet II, Src: Intel8026:fe (78:ad:d3:1a:d8:fe), Dst: Hewlett-Packard:3a:tc:96 (20:52:4c:3a:tc:96)
* Internet Protocol Version 4, Src: 172.17.0.253, Dst: 172.17.0.1
* User Datagram Protocol, Src Port: 54378, Dst Port: 53
* Domain Name System (query)

0000  20 5f 44 24 0c 00 70 e8  23 7a 88 f6 00 30 42 00  |...g...t
0010  00 30 28 27 24 00 40 11  78 07 0c 11 82 f0 0c 11  |...y...t
0020  00 01 46 4a 04 20 00 20  1b 5a e5 0a 01 30 00 01  |...s)...
0030  00 00 00 00 00 00 00 75  0f 72 72 72 77 09 07 07  |...network
0040  05 77 03 0e 04 74 00 00  01 00 00 00 00 00 00 00  |...er net-

Metadata:
  Seq: 2, Size: 2, Offset: 0, Source: 172.17.0.253, Destination: 172.17.0.1, Protocol: DNS, Length: 75, Info: Standard query (id 54378)
  Show packet bytes  Layout: Vertical (stacked)
  Help  Close
```

```
Wireshark - Packet 40 - eth0

* Frame 40: 156 bytes on wire (1248 bits), 156 bytes captured (1248 bits) on interface wlan0, id 0
* Ethernet II, Src: Hewlett-Packard:3a:tc:96 (20:52:4c:3a:tc:96), Dst: Intel8026:fe (78:ad:d3:1a:d8:fe)
* Internet Protocol Version 4, Src: 172.17.0.1, Dst: 172.17.0.253
* User Datagram Protocol, Src Port: 53, Dst Port: 54378
* Domain Name System (response)

0000  78 ad d3 1a d8 fe 20 12  4a 7a c6 06 00 00 45 00  |...E
0010  00 00 00 00 00 00 00 1a  72 12 00 11 00 00 00 11  |...
0020  0f f0 8a 35 c3 a3 00 1a  0c 4f 8e 0b 83 00 00 01  |...S...r...
0030  00 00 00 01 00 00 00 70  0f 72 72 71 77 09 07 07  |...network
0040  0a 72 03 0e 04 74 00 00  01 00 00 00 00 00 00 00  |...er net-...
0050  01 00 00 03 00 00 45 00  06 72 00 37 23 09 01 00  |...E...573...
0060  7f 73 04 04 f3 00 30 17  03 01 11 60 77 73 04 00  |...DNS...
0070  71 70 40 6f 72 72 00 41  72 1a 00 71 00 85 00 01  |...network...
0080  7a 07 00 02 02 07 00 00  00 00 00 00 00 00 00 00  |...
0090  00 00 03 04 00 12 75 00  00 01 51 00 00 00 00 00  |...

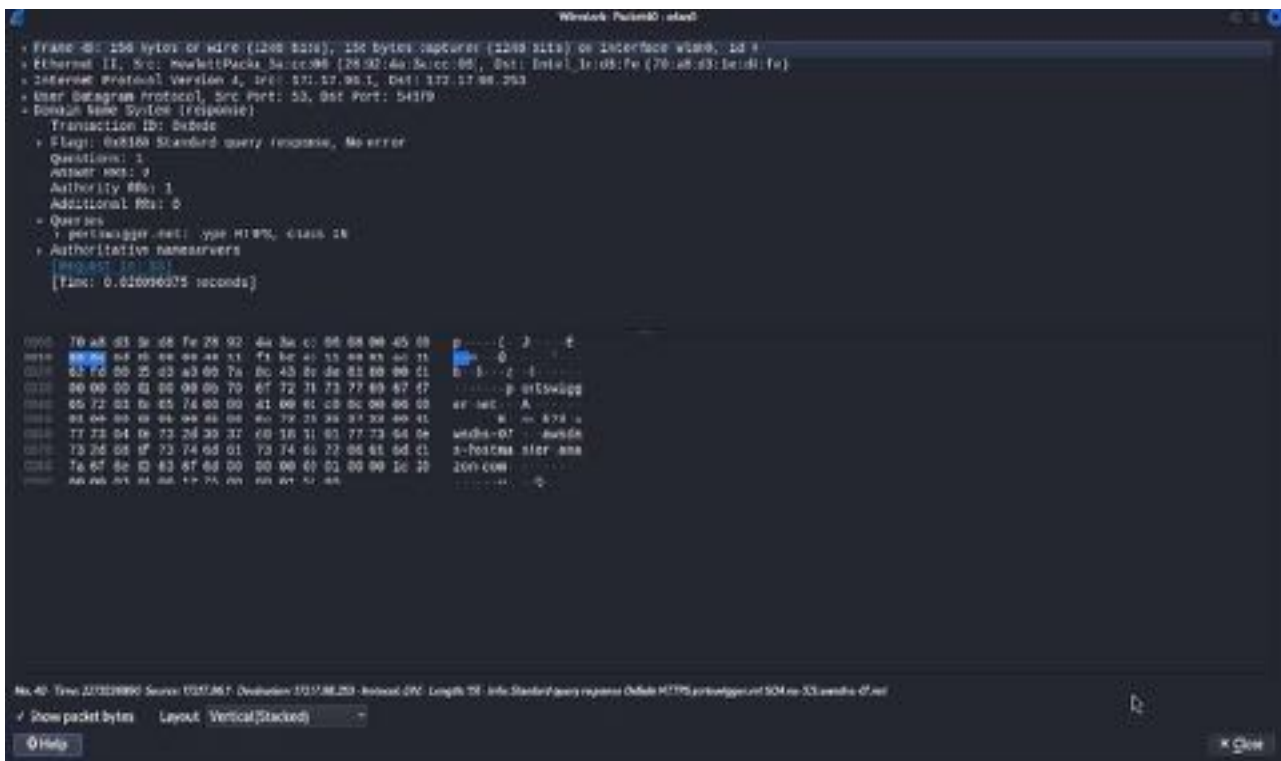
Metadata:
  Seq: 2, Size: 2, Offset: 0, Source: 172.17.0.1, Destination: 172.17.0.253, Protocol: DNS, Length: 156, Info: Standard query response (id 54378)
  Show packet bytes  Layout: Vertical (stacked)
  Help  Close
```

Question 6: To what IP address is the DNS query message sent? Use [Kali tool] to determine the IP address of your local DNS server. Are these two IP addresses the same?

Ans A computer is configured with a "local DNS server" (or resolver) to send all its DNS queries. The resolv.conf file in Linux stores the IP address of this server.

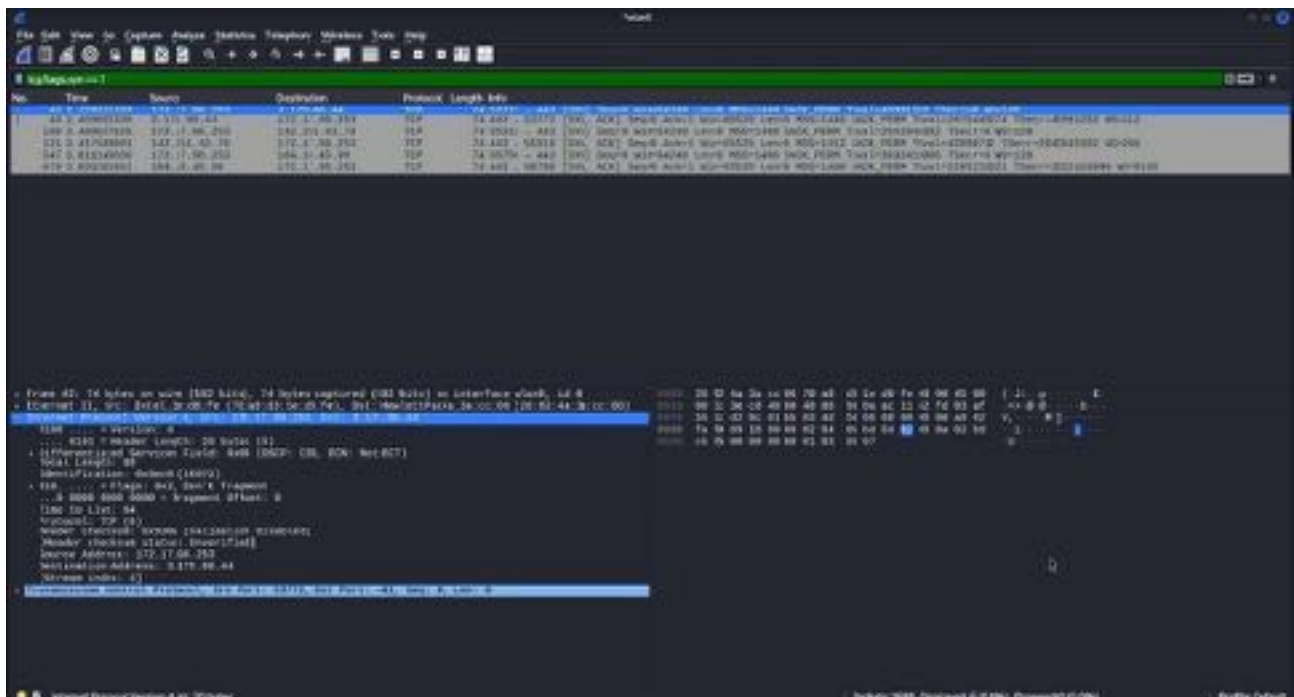
Question 8: Examine the DNS response message. How many "answers" are provided? What does each of these answers contain?

Ans The DNS response message provides the answer(s) to the query. The "Answer RRs" field will be 1 (or more), and the "Answers" section will contain the actual resource record, which includes the requested name, its type, and the corresponding data (e.g., the IP address).



Question 9: Consider the subsequent TCP SYN packet sent by your host. Does the destination IP address of the SYN packet correspond to any of the IP addresses provided in the DNS response message?

Ans This is the key concept. After the host receives the IP address from DNS (the answer), it *then* initiates a **TCP connection** to that exact IP to fetch the webpage. A TCP connection begins with a SYN (synchronize) packet.



Question 10: This web page contains images. Before retrieving each image, does your host issue new DNS queries?

Ans A web page and its embedded objects (like images, scripts, ads) can be hosted on different domains. If an image is on a separate domain (e.g., images.ietf-cdn.com), the browser must perform a *new* DNS query for that domain to get its IP before it can download the image.

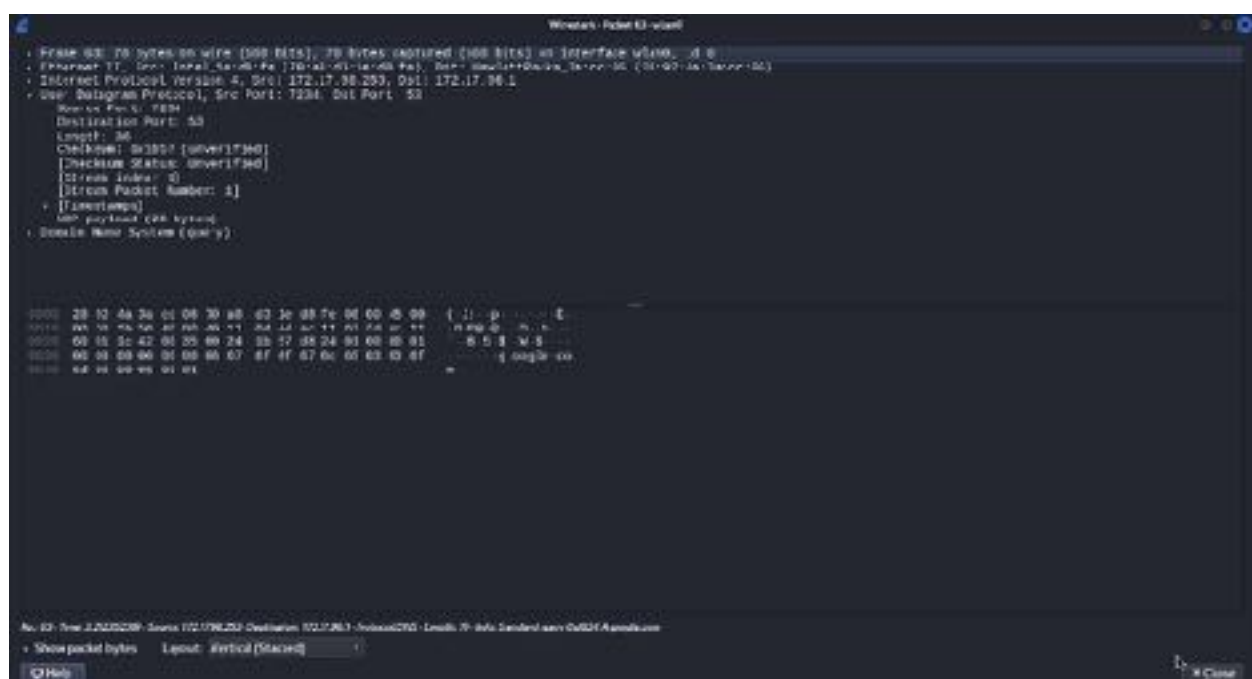


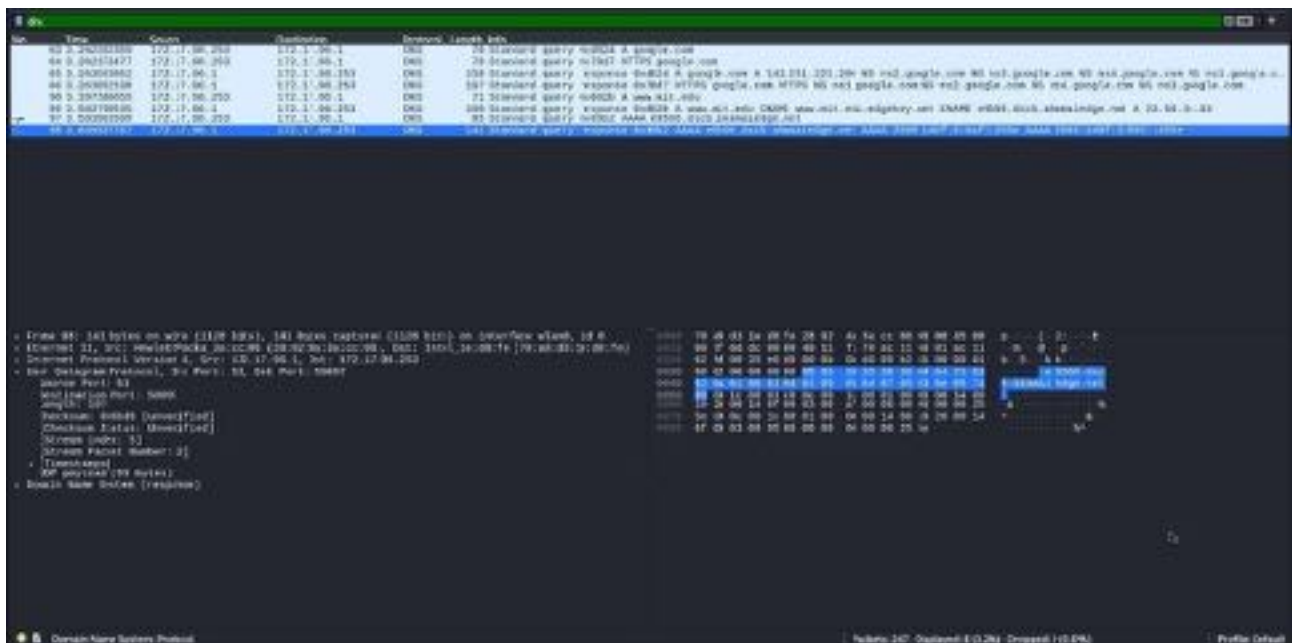
```
(kali@kali) [~]
$ nslookup www.mit.edu
Server:           172.17.0.1
Address:          172.17.0.1#53

Non-authoritative answer:
www.mit.edu       canonical name = www.mit.edu.edgekey.net.
www.mit.edu.edgekey.net canonical name = e9566.dsca.akamaiedge.net.
Name:   e9566.dsca.akamaiedge.net
Address: 23.58.34.33
Name:   e9566.dsca.akamaiedge.net
Address: 2600:140f:3:895::255e
Name:   e9566.dsca.akamaiedge.net
Address: 2600:140f:3:8af::255e

(kali@kali) [~]
$
```

Question 11: What is the destination port for the DNS query message? What is the source port of DNS response message?





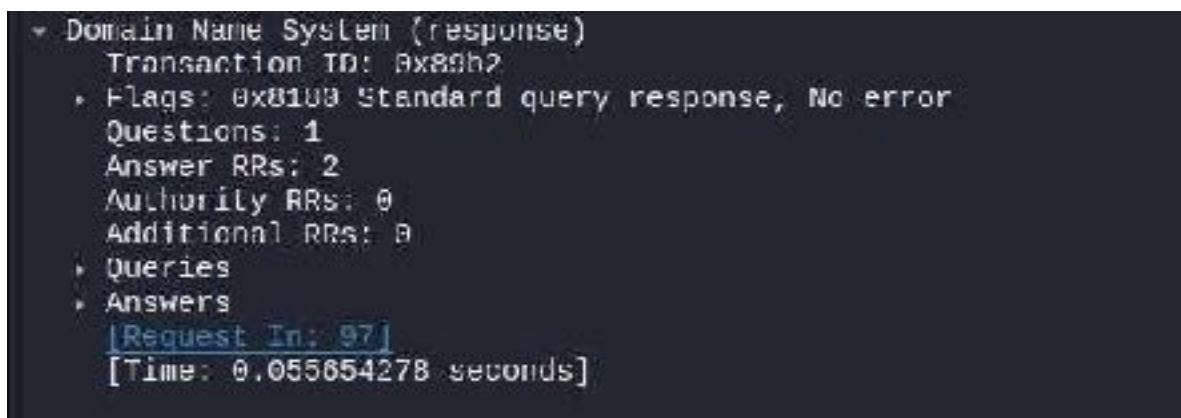
Ans The standard DNS port is 53.

Question 12: To what IP address is the DNS query message sent? Is this the IP address of your default local DNS server?

Ans By default, nslookup(like any application) uses the system's configured local DNS server.

Question 13: Examine the DNS query message. What "Type" of DNS query is it? Does the query message contain any "answers"?

Ans nslookup's default query is for an 'A' record.



Question 14: Examine the DNS response message. How many "answers" are provided? What does each of these answers contain?

Ans The response will contain the 'A' record for www.mit.edu

```

+ Frame 65: 187 bytes on wire (1336 bits), 187 bytes captured (1336 bits) on interface wlan0, id 8
+ Ethernet II, Src: Hewlett-Packard_08:00:27:14:00:08, Dst: Intel_Ethernet (78:ad:d3:1e:d8:fc)
+ Internet Protocol Version 4, Src: 172.17.0.1, Dst: 172.17.0.253
+ User Datagram Protocol, Src Port: 52, Dst Port: 27243
  Source Port: 52
  Destination Port: 27243
  Length: 133
  Checksum: 8ae94b [unverified]
  [Checksum Status: Unverified]
  [Stream Index: 2]
  [Stream Packet Number: 2]
  [Timestamps]
  UDP payload (125 bytes)
+ Domain Name System (response)
  Transaction ID: 0x78d7
  Flags: 0x8180 Standard query response, No error
  Questions: 1
  Answer RRs: 1
  Authority RRs: 4
  Additional RRs: 0
  Queries
  Answers
  Authoritative Name Servers
  [Request ID: 64]
  [Time: 0.087518721 seconds]

```

Domain Name System: Protocol

No.	Time	Source	Destination	Protocol	Length	Info
64	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
65	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
66	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
67	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
68	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
69	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
70	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
71	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
72	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
73	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
74	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
75	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
76	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
77	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
78	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
79	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
80	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
81	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
82	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
83	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
84	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
85	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
86	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
87	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
88	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
89	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
90	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
91	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
92	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
93	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
94	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
95	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
96	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
97	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
98	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com
99	0.26221477	172.17.0.1	172.17.0.1	DNS	79	Standard query request to google.com

Question 16: To what IP address is the DNS query message sent? Is this the IP address of your default local DNS server?

Ans Even with a different type, nslookup still uses the default server.

```

+ Frame 63: 234 bytes on wire (1872 bits), 234 bytes captured (1872 bits) on interface wlan0, id 0
+ Ethernet II, Src: Hewlett-Packard_3a:4a:00:26:02:4a, Dst: Intel_Ethernet (78:ad:d3:1e:d8:fc)
+ Internet Protocol Version 4, Src: 172.17.0.1, Dst: 172.17.0.253
  0100 .... = Version: 4
  ... 0101 = Header Length: 20 bytes (5)
+ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 220
  Identification: 0xcdf (29145)
+ 000 .... = Flags: 0x0
  ... 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 64
  Protocol: UDP (17)
  Header Checksum: 0xf0fe [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 172.17.0.1
  Destination Address: 172.17.0.253
  [Stream Index: 3]
+ User Datagram Protocol, Src Port: 52, Dst Port: 34529
+ Domain Name System (response)

```

Domain Name System: Protocol

Question 17: Examine the DNS query message. What "Type" of DNS query is it? Does the query message contain any "answers"?

Ans This time, the query is *explicitly* asking for an 'NS' record, as specified by the -type=NS command. It still contains no answers.

```
Frame 63: 234 bytes on wire (1872 bits), 234 bytes captured (1872 bits) on interface wlan0, id 0
Ethernet II, Src: HewlettPacka_3a:cc:86 (28:92:4a:3a:cc:86), Dst: Intel_1e:d8:fe (78:a8:d3:1e:d8:fe)
Internet Protocol Version 4, Src: 172.17.96.1, Dst: 172.17.96.253
User Datagram Protocol, Src Port: 53, Dst Port: 34529
Domain Name System (query)
Transaction ID: 0x88f3
Flags: 0x0100 Standard query response, No error
Questions: 1
Answer RRs: 0
Authority RRs: 0
Additional RRs: 0
Queries
  mit.edu: type NS, class IN
Answers
  [Request In: 62]
  [Time: 0.826957810 seconds]
```

Domain Name System: Protocol

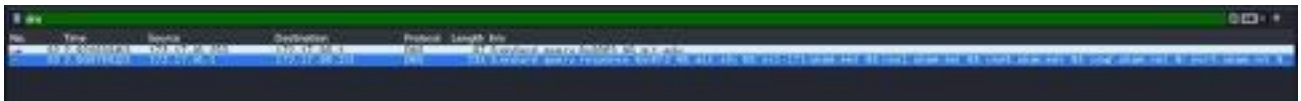
Question 18: Examine the DNS response message. What MIT name servers does the response message provide? Does this response message also provide the IP addresses of the MIT name servers?

Ans The response will provide the 'NS' records in the "Answers" section. To be helpful, DNS servers often include the 'A' records (IP addresses) for those name servers in the "Additional records" section. This saves the client from having to make *another* query to find the IP of the name server.

```
Frame 63: 234 bytes on wire (1872 bits), 234 bytes captured (1872 bits) on interface wlan0, id 0
Ethernet II, Src: HewlettPacka_3a:cc:86 (28:92:4a:3a:cc:86), Dst: Intel_1e:d8:fe (78:a8:d3:1e:d8:fe)
Internet Protocol Version 4, Src: 172.17.96.1, Dst: 172.17.96.253
User Datagram Protocol, Src Port: 53, Dst Port: 34529
Domain Name System (response)
Transaction ID: 0x88f3
Flags: 0x0100 Standard query response, No error
Questions: 1
Answer RRs: 0
Authority RRs: 0
Additional RRs: 0
Queries
  mit.edu: type NS, class IN
Answers
  [Request In: 62]
  [Time: 0.826957810 seconds]
```

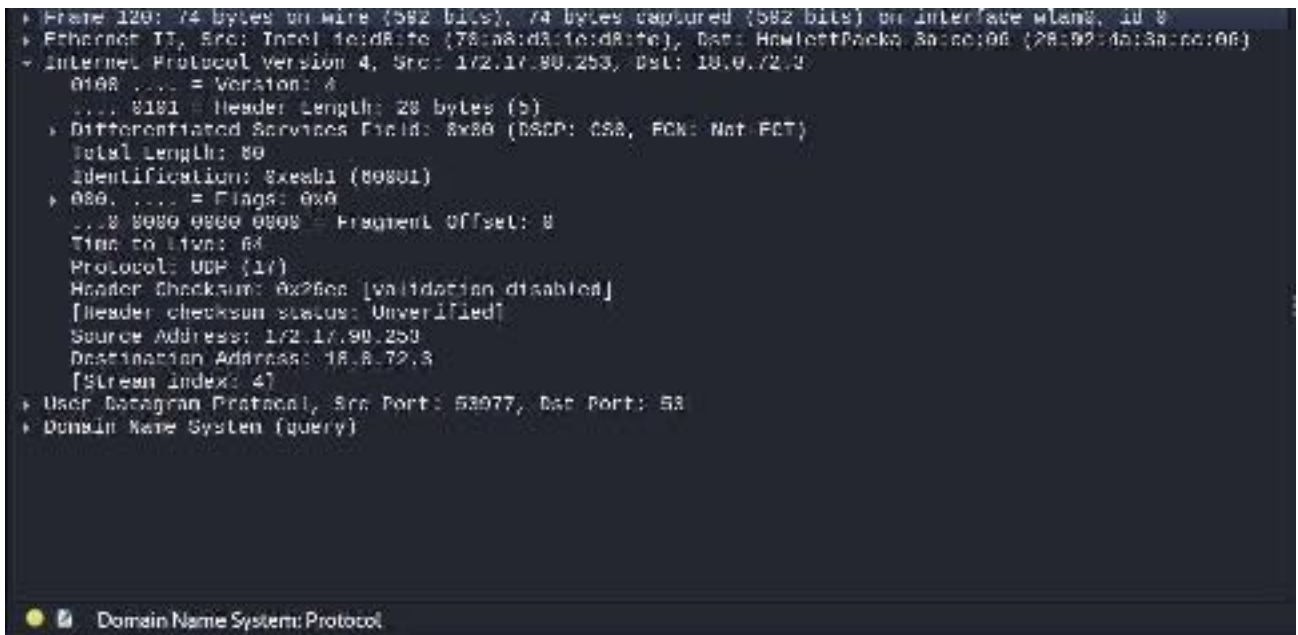
Question 19:

Ans



Question 20: To what IP address is the DNS query message sent? Is this the IP address of your default local DNS server? If not, what does the IP address correspond to?

Ans in this command, we *explicitly* told nslookup to query the server bitsy.mit.edu. Therefore, the query message is *not* sent to our local DNS server, but directly to the IP address of bitsy.mit.edu.



Question 21: Examine the DNS query message. What "Type" of DNS query is it? Does the query message contain any "answers"?

Ans We are asking for the IP of www.aiit.or.kr, so the query type is 'A'.

```

+ Frame 120: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface wlan0, id 0
+ Ethernet II, Src: Intel_1e:d8:fe (78:a8:d3:1e:d8:fe), Dst: HewlettPacka_3a:cc:06 (28:92:4a:3a:cc:06)
+ Internet Protocol Version 4, Src: 172.17.0.253, Dst: 10.0.72.3
+ User Datagram Protocol, Src Port: 53977, Dst Port: 53
+ Domain Name System (query)
  Transaction ID: 0x101a
  + Flags: 0x0100 Standard query
    Questions: 1
    Answer RRs: 0
    Authority RRs: 0
    Additional RRs: 0
  - Queries
    + www.aiit.or.kr: type AAAA, class IN
    [Response In: 131]

```

Domain Name System: Protocol

Question 22: Examine the DNS response message. How many "answers" are provided? What does each of these answers contain?

Ans The response from bitsy.mit.edu will contain the 'A' record for www.aiit.or.kr.

```

+ Frame 131: 130 bytes on wire (1040 bits), 130 bytes captured (1040 bits) on interface wlan0, id 0
+ Ethernet II, Src: HewlettPacka_3a:cc:06 (28:92:4a:3a:cc:06), Dst: Intel_1e:d8:fe (78:a8:d3:1e:d8:fe)
+ Internet Protocol Version 4, Src: 10.0.72.3, Dst: 172.17.0.253
+ User Datagram Protocol, Src Port: 53, Dst Port: 53977
+ Domain Name System (response)
  Transaction ID: 0x101a
  + Flags: 0x8100 Standard query response, No error
    Questions: 1
    Answer RRs: 2
    Authority RRs: 0
    Additional RRs: 0
  - Queries
    + www.aiit.or.kr: type AAAA, class IN
  - Answers
    + www.aiit.or.kr: type AAAA, class IN, addr 2606:4700:3036::6815:4a08
    + www.aiit.or.kr: type AAAA, class IN, addr 2606:4700:3031::ac43:9878
    [Request In: 120]
    [Time: 0.408575275 seconds]

```

Domain Name System: Protocol

Question 23:

Ans

No.	Time	Source	Destination	Protocol	Length	Info
100	0.0000000	192.17.0.100	192.17.0.1	ICMP	28	Standard query request 0x100 0x100 0x100
101	0.0000000	192.17.0.100	192.17.0.1	ICMP	28	Standard query request 0x100 0x100 0x100
102	0.0000000	192.17.0.1	192.17.0.100	ICMP	84	Standard query response 0x100 0x100 0x100 0x100 0x100 0x100
103	0.0000000	192.17.0.1	192.17.0.100	ICMP	84	Standard query response 0x100 0x100 0x100 0x100 0x100 0x100
104	0.0000000	192.17.0.100	192.17.0.1	ICMP	28	Standard query request 0x100 0x100 0x100
105	0.0000000	192.17.0.1	192.17.0.100	ICMP	84	Standard query response 0x100 0x100 0x100 0x100 0x100 0x100
106	0.0000000	192.17.0.100	192.17.0.1	ICMP	28	Standard query request 0x100 0x100 0x100
107	0.0000000	192.17.0.1	192.17.0.100	ICMP	84	Standard query response 0x100 0x100 0x100 0x100 0x100 0x100

Lab EX 3

Ping is a tool used to test network connectivity. It uses the **ICMP (Internet Control Message Protocol)**.

- It works by sending an **"ICMP Echo Request"** packet to a destination IP. If the destination is reachable, it replies with an **"ICMP Echo Reply"** packet.
- Encapsulation:** This exercise shows the concept of layers. The ICMP message (Innermost) is "wrapped" inside an **IP Header** (Middle), which is then "wrapped" inside an **Ethernet Header** (Outermost) to be sent on the local network.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.0000000	192.17.0.100	192.17.0.1	ICMP	28	Standard query request 0x100 0x100 0x100
2	0.0000000	192.17.0.100	192.17.0.1	ICMP	28	Standard query request 0x100 0x100 0x100
3	0.0000000	192.17.0.1	192.17.0.100	ICMP	84	Standard query response 0x100 0x100 0x100 0x100 0x100 0x100

1. What is the IP address of the host being pinged?

No.	Time	Source	Destination	Protocol	Length	Info
1	0.0000000	192.17.0.100	192.17.0.1	ICMP	28	Standard query request 0x100 0x100 0x100
2	0.0000000	192.17.0.100	192.17.0.1	ICMP	28	Standard query request 0x100 0x100 0x100
3	0.0000000	192.17.0.1	192.17.0.100	ICMP	84	Standard query response 0x100 0x100 0x100 0x100 0x100 0x100

2. What are the 3 layers in packet 1, starting with the outermost?

2. What port does the client run on?

```
Frame 1: 78 bytes on wire (624 bits), 78 bytes captured (624 bits) on interface en0, id 0
Ethernet II, Src: Apple_98:88:89 (4c:32:75:98:88:89), Dst: IETF-VRRP-VRID_43 (00:00:5e:00:01:43)
Internet Protocol Version 4, Src: 10.35.3.213, Dst: 171.64.13.18
Transmission Control Protocol, Src Port: 51319, Dst Port: 25, Seq: 0, Len: 0
```

3. What protocol does SMTP run on top of?

```
Frame 4: 115 bytes on wire (920 bits), 115 bytes captured (920 bits) on interface en0, id 0
Ethernet II, Src: JuniperNetw_fa:3c:9b (28:98:00:fa:3c:9b), Dst: Apple_98:88:89 (4c:32:75:98:88:89)
Internet Protocol Version 4, Src: 171.64.13.18, Dst: 10.35.3.213
Transmission Control Protocol, Src Port: 25, Dst Port: 51319, Seq: 1, Ack: 1, Len: 40
Single-Byte Text Transfer Protocol
Response: 220 smtp-unencrypted.stanford.edu SMTP Postfix/3.1.0
Response code: <domain> Service ready [220]
Response parameter: smtp-unencrypted.stanford.edu SMTP Postfix

0000  4c 32 75 98 88 89 28 08 9b fa 3c 49 48 09 45 00  L24-----I-E-
0010  00 00 07 05 00 00 f8 00 32 03 40 49 00 12 00 23  +.8...2.0...e
0020  03 02 00 00 c9 77 75 fb a3 2e 40 19 fc 0f 00 30  ....WA.....
0030  M Fx jk w w v v A v A 00 00 00 00 00 40 c0 40  ...V.....A/V
0040  59 43 32 31 30 20 73 00 74 70 20 75 00 05 00 03  Y120 sm tp-unenc
0050  72 79 78 74 05 04 2e 73 74 01 0e 06 0f 72 04 3e  ryped.s stanford.
0060  65 04 75 20 45 03 60 54 50 20 50 0f 73 78 00 09  ed:SMTP Postfix
0070  78 04 04
```

No. 4 - Time: 0.016420 - Source: 171.64.13.18 - Destination: 10.35.3.213 - Protocol: SMTP - Length: 115 - Info: 5 220 smtp-unencrypted.stanford.edu SMTP Postfix

✓ Show packet bytes Layout: Vertical [Stacked] -

Help

Close

4 a. Which packet represents the telnet request?

Ans The "telnet request" isn't a *real* telnet packet. It's just initiating a simple TCP connection, which is what Telnet does

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.35.3.213	171.64.13.18	TCP	78	51510 → 25 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=997611847 TSecr=0 SACK_PERM
2	0.002266	171.64.13.18	10.35.3.213	TCP	74	25 → 51510 [SYN, ACK] Seq=0 Ack=1 Win=0 Len=0 MSS=1300 TSval=2369281143 TSecr=997611847 SACK_PERM
3	0.002348	10.35.3.213	171.64.13.18	TCP	66	51510 → 25 [ACK] Seq=1 Ack=1 Win=65535 Len=0 TSval=997611849 TSecr=2369281143

4

b. Which packet contains the 220 response from the SMTP server?

[illegible]

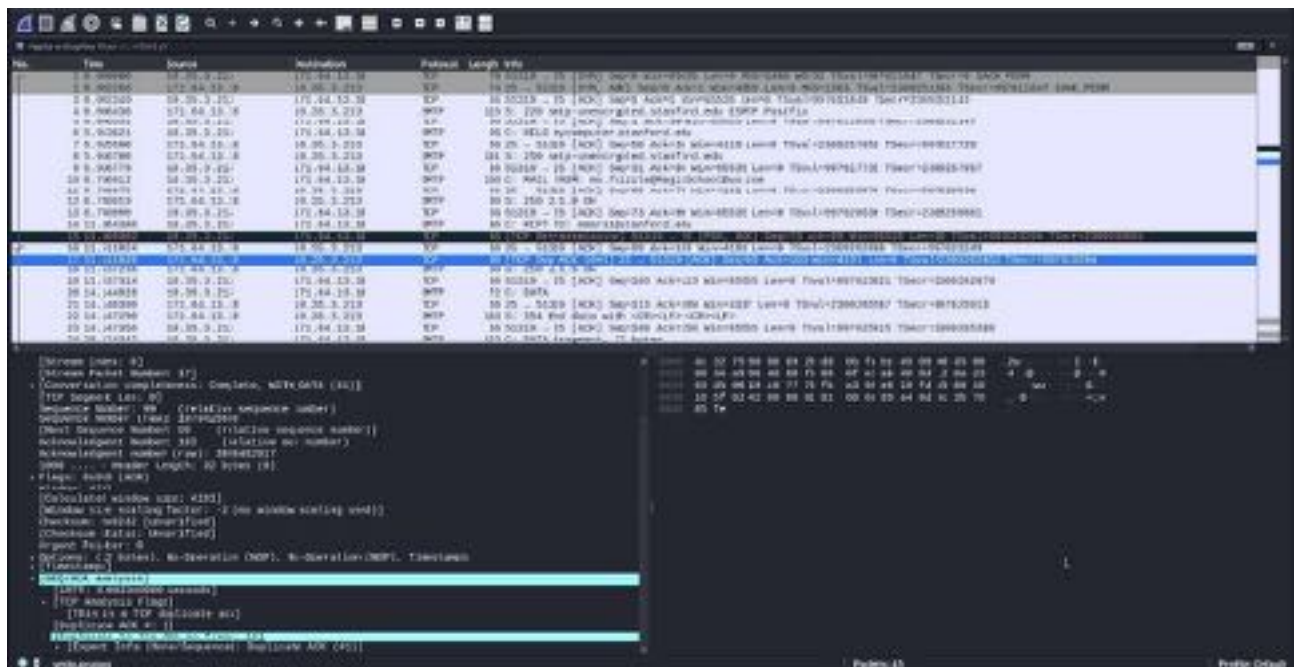
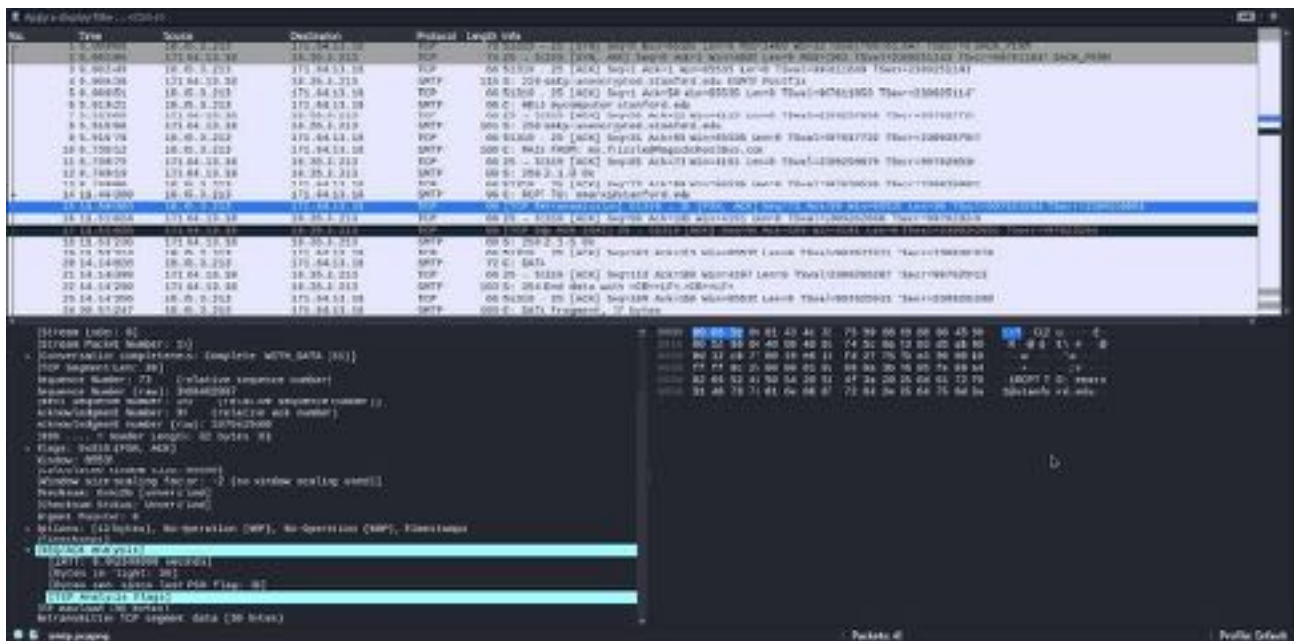
a. In which packet does the client first acknowledge the 220?

[illegible]

5. d. What data was in the byte the client sent before the 220?

Ans The client's sequence number is 1 because it already "spent" one byte.

[illegible]



Traceroute is a tool used to map the network path (the "hops") between your computer and a destination.

- It works by sending packets (in this case, UDP packets) with a limited **Time-To-Live (TTL)**.
- It sends the first packet with TTL=1. The first router (hop 1) receives it, decrements TTL to 0, and sends back an **ICMP "Time-to-Live exceeded"** error.
- It then sends a packet with TTL=2. The second router (hop 2) sends back the ICMP error.
- It repeats this, increasing the TTL each time, until the packets reach the final destination. The ICMP error messages reveal the IP address of each router along the path.

The packet of trace route is not present

1. a. What is the IP of the host requesting the traceroute?

- **Step 1:** Look at Packet 5 (the first packet to ignore the note).
- **Step 2:** In "Packet Details," expand Internet Protocol Version 4.
- **Step 3:** The **Source:**IP address is your host.

1. b. How does the sending host determine MIT's IP address?

- **Step 1:** Look closely at **Packets 5-8**.
- **Step 2:** You will see a **DNS query** for mit.edu(Packet 5) and a **DNS response** (Packet 8) that provides its IP address.

2. a. What is the innermost protocol of these packets?

- **Step 1:** Click on Packet 9 (the first packet *after* the DNS lookup).
- **Step 2:** Look at the layers in the "Packet Details" pane. The innermost protocol is **User Datagram Protocol (UDP)**.

2. b. How many packets does it send before getting the first response? ... What is the TTL of the last packet sent?

- **Step 1:** Look at the trace. Packet 9 is the first UDP probe. Packet 10 is the ICMP response.
- **Answer (Packets):** It sends **one packet** (Packet 9).
- **Answer (TTL):** Click on Packet 9. In the Internet Protocol Version 4 details, the **Time to live:** will be **1**.

2. c. What do you notice about the source and destination ports...?

- **Step 1:** Look at the UDP packets being sent (Packet 9, 11, 13, etc.).
- **Step 2:** Expand the User Datagram Protocol layer for each.
- **Answer:** The **Source Port** is the same (or increments), and the **Destination Port** increments for each packet (e.g., 33434, 33435, 33436...).

2. d. Which packet is the first response responding to?

- **Step 1:** Click on Packet 10 (the first ICMP response).
- **Step 2:** In "Packet Details," expand Internet Control Message Protocol.
- **Step 3:** Inside, you'll see it contains the IP and UDP headers of the original packet that triggered it.
- **Step 4:** Look at the User Datagram Protocol *inside* this ICMP packet. The destination port will match the destination port of **Packet 9**.

3. Look at the traceroute output for hops 10, 13, and 15. What is different?

- **Step 1:** Look at the text output provided in the PDF .

- **Step 2:**

- Hop 10 shows *two different IPs* responding (203.208.149.250 and 203.208.172.233).
- Hop 13 shows a different IP (203.208.190.138).
- Hop 15 shows *two different IPs* (210.49.108.62 and 210.49.112.114).

- **Answer:** These hops show **multiple different IP addresses** responding for the same TTL, which indicates a load-balanced or redundant path.

4. a. Which packets did the source send to prompt the responses from hop 10?

- **Theory:** The PDF has a hint that the 2nd hop *responds* to TTL=1. This means Hop 10 (the 10th hop listed) will be responding to packets with **TTL=9**.
- **Step 1:** In the Wireshark display filter, type `ip.ttl == 9` and press Enter.
- **Step 2:** You will see the three UDP packets that were sent to probe hop 10.

4. b. Subtract the timestamp of the packet sent to host 203.208.172.233 from the timestamp of the corresponding response.

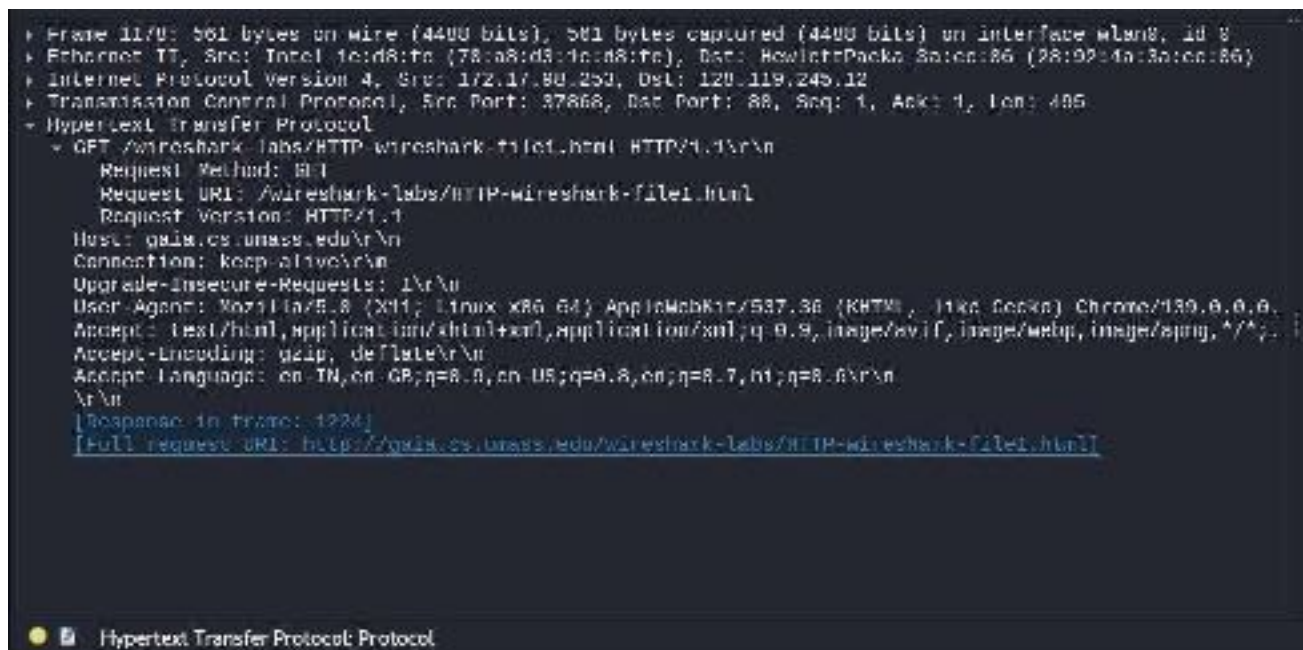
- **Step 1:** From your filtered list in Q4a, find the packet sent to 203.208.172.233. (You'll have to look at the ICMP *responses* to see which probe this IP responded to, *then* find the probe).
 - *Easier way:* The text output shows 203.208.172.233 responded to the *third* probe for hop 10.
 - Find the third packet in your `ip.ttl == 9` filtered list. Click it and note its timestamp in the **Time** column.
- **Step 2:** Clear the filter. Find the ICMP response *from* 203.208.172.233. Note its timestamp.
- **Step 3:** Subtract (Time of Response) - (Time of Probe).
- **Step 4:** The result (e.g., 0.012...seconds) will be **12.x milliseconds**, matching the 12.462 ms reported in the text.

Ex 3.1 Wireshark HTTP

HTTP (Hypertext Transfer Protocol): An application-layer protocol for transmitting web content (like HTML pages, images, etc.). It's a "client-server" protocol, where the client (your browser) sends a **request** (e.g., GET) and the server (hosting the website) sends a **response** (e.g., 200 OK).

- **HTTP Messages:** Both requests and responses are plain text and consist of:
 1. **Start Line:** Specifies the request (e.g., GET /page.html HTTP/1.1) or the response status (e.g., HTTP/1.1 200 OK).
 2. **Headers:** A list of Key: Value pairs providing metadata (e.g., Host: www.example.com).
 3. **Blank Line:** Separates headers from the body.
 4. **Body (Optional):** The actual content (e.g., the HTML of the page or data sent to a server).
- **Underlying Protocol:** Like SMTP, HTTP runs on top of **TCP** to ensure reliable, in-order delivery of all the data.

1. Is your browser running HTTP version 1.0 or 1.1? What version of HTTP is the server running?



The image shows a Wireshark packet capture of an HTTP GET request. The packet list on the left shows a packet of 561 bytes on wire (4480 bits) captured on interface wlan0. The packet details pane shows the following structure:

- Ethernet II, Src: Intel 16:d8:1e (78:a8:d3:16:d8:1e), Dst: Hewlett-Packard 28:92:4a:3a:0c:86
- Internet Protocol Version 4, Src: 172.17.0.253, Dst: 129.119.245.12
- Transmission Control Protocol, Src Port: 37868, Dst Port: 80, Seq: 1, Ack: 1, Len: 485
- Hypertext Transfer Protocol
 - GET /wireshark-labs/HTTP-wireshark-file1.html HTTP/1.1
 - Request Method: GET
 - Request URI: /wireshark-labs/HTTP-wireshark-file1.html
 - Request Version: HTTP/1.1
 - Host: gaia.cs.umass.edu
 - Connection: keep-alive
 - Upgrade-Insecure-Requests: 1
 - User-Agent: Mozilla/5.0 (X11; Linux x86_64; AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.0.0)
 - Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/svg+xml,*/*;q=0.8
 - Accept-Encoding: gzip, deflate
 - Accept-Language: en-IN,en-GB;q=0.8,en-US;q=0.8,en;q=0.7,hi;q=0.6

The status bar at the bottom indicates the protocol is Hypertext Transfer Protocol.

2. What languages (if any) does your browser indicate that it can accept to the server?

```

+ Frame 1170: 561 bytes on wire (4496 bits), 561 bytes captured (4496 bits) on interface wlan0, id 8
+ Ethernet II, Src: Intel 1c:d8:fc (78:a8:d3:1c:d8:fc), Dst: Hewlett-Packard 3a:cn:86 (28:92:4a:3a:cn:86)
+ Internet Protocol Version 4, Src: 172.17.90.253, Dst: 128.119.245.12
+ Transmission Control Protocol, Src Port: 37888, Dst Port: 88, Seq: 1, Ack: 1, Len: 495
+ Hypertext Transfer Protocol
  - GET /wireshark-labs/HTTP-wireshark-file1.html HTTP/1.1\r\n
    Request Method: GET
    Request URI: /wireshark-labs/HTTP-wireshark-file1.html
    Request Version: HTTP/1.1
    Host: gaia.cs.umass.edu\r\n
    Connection: keep-alive\r\n
    Upgrade-Insecure-Requests: 1\r\n
    User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.0.0
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
    Accept-Encoding: gzip, deflate\r\n
    Accept-Language: en-IN,en-GB;q=0.8,en-US;q=0.6,en;q=0.7,hi;q=0.6\r\n
    \r\n
    [Response in frame: 1234]
    [Full request URI: http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html]

HTTP Accept Encoding (http.accept_encoding), 32 bytes

```

3. What is the IP address of your computer? Of the gaia.cs.umass.eduserver?

```

+ Frame 1170: 561 bytes on wire (4496 bits), 561 bytes captured (4496 bits) on interface wlan0, id 8
+ Ethernet II, Src: Intel 1c:d8:fc (78:a8:d3:1c:d8:fc), Dst: Hewlett-Packard 3a:cn:86 (28:92:4a:3a:cn:86)
+ Internet Protocol Version 4, Src: 172.17.90.253, Dst: 128.119.245.12
  0108 ..... = Version: 4
  .... 8181 = Header Length: 20 bytes (5)
+ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 547
  Identification: 0x47bb (10383)
+ 010 ..... = Flags: 0x2, Don't fragment
  ... 8080 8080 0800 = Fragment Offset: 0
  Time to live: 64
  Protocol: TCP (6)
  Header Checksum: 0x6c87 [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 172.17.90.253
  Destination Address: 128.119.245.12
  [Stream Index: 24]
+ Transmission Control Protocol, Src Port: 37888, Dst Port: 88, Seq: 1, Ack: 1, Len: 495
+ Hypertext Transfer Protocol

HTTP Accept Encoding (http.accept_encoding), 32 bytes

```

4. What is the status code returned from the server to your browser?

No.	Time	Source	Destination	Protocol	Length	Info
1178	17.70250410	172.17.0.253	172.17.0.253	HTTP	501	SET /wirespark-labs/HTTP-wirespark-file.html HTTP/1.1
1179	17.70250450	172.17.0.253	172.17.0.253	HTTP	555	HTTP/1.1 200 OK (text/html)
1229	26.00790016	172.17.0.253	172.17.0.253	HTTP	507	SET /favicon.ico HTTP/1.1
1243	26.27481964	172.17.0.253	172.17.0.253	HTTP	540	HTTP/1.1 404 NOT FOUND (text/html)

- Frame 1178: 501 bytes on wire (4048 bits), 501 bytes captured (4048 bits) on interface wlan0, id 0 - Ethernet II, Src: IntelE1000e (78:1d:63:2e:56:fe), Dst: RealtekPcs_3a:00:00 (28:92:4a:34:0c:10) - Internet Protocol Version 4, Src: 172.17.0.253, Dst: 172.17.0.253 - Transmission Control Protocol, Src Port: 37688, Dst Port: 80, Seq: 1, Ack: 1, Len: 405 - Hypertext Transfer Protocol - SET /wirespark-labs/HTTP-wirespark-file.html HTTP/1.1	<pre> 0000 97 4e 47 45 54 20 1f 77 04 72 85 73 68 61 72 6b - 1 0010 2d 5c 83 62 72 27 4b 34 04 50 26 17 09 17 05 13 - 1 0020 60 14 78 86 7e 66 4b 6e 81 91 7e 88 74 84 6e 90 - 1 0030 48 14 54 58 2f 31 3e 31 00 0a 48 6f 73 11 3a 20 - 1 0040 67 15 89 65 28 63 12 28 75 60 61 73 73 25 65 64 - 1 0050 75 64 0a 82 6f 56 6e 65 81 74 80 6f 60 70 20 60 - 1 0060 85 06 06 24 83 6e 16 16 26 6d 64 55 70 62 12 61 - 1 0070 64 15 20 49 6e 71 15 83 71 72 85 28 52 65 71 73 - 1 0080 65 73 74 73 3a 20 11 88 8a 55 73 85 72 21 41 67 - 1 0090 65 6e 74 3a 20 40 4f 7a 68 6c 5c 61 2f 10 3e 30 - 1 0100 20 15 58 31 31 3e 40 80 60 75 78 20 73 18 35 1 - 1 0110 5f 18 3a 20 20 21 16 70 8c 65 53 85 83 45 60 72 - 1 0120 2f 15 33 37 2e 33 10 20 21 40 40 54 4d 4c 2c 29 - 1 0130 60 15 00 85 20 47 15 83 80 8f 20 20 43 60 72 6f - 1 0140 6d 15 2f 15 33 20 20 21 30 20 20 20 5a 61 60 - 1 0150 01 12 69 2f 33 33 17 20 21 30 60 0a 41 60 63 65 - 1 0160 70 14 3a 20 74 85 18 74 2f 68 74 80 60 21 61 70 - 1 </pre>
--	---

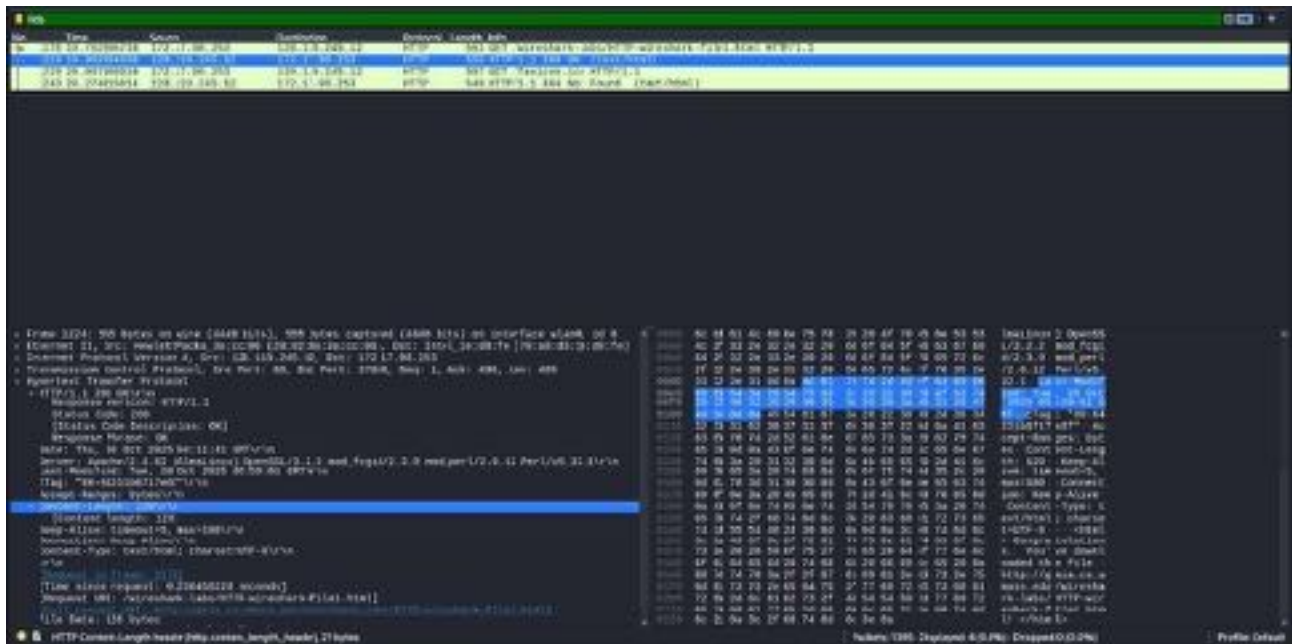
Request Method: GET
 Request URL: /wirespark-labs/HTTP-wirespark-file.html
 Request Version: HTTP/1.1
 Host: pacs.cs.umass.edu/r/n
 Connection: keep-alive/r/n
 Upgrade-Insecure-Requests: 1/r/n
 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/531.36 (KHTML, like Gecko) Chrome/139.0.6.0.5
 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8
 Accept-Encoding: gzip, deflate/r/n
 Accept-Language: en-ME,en-gb;q=0.9,en-US;q=0.8,en;q=0.7,hi;q=0.5/r/n

5. When was the HTML file that you are retrieving last modified at the server?

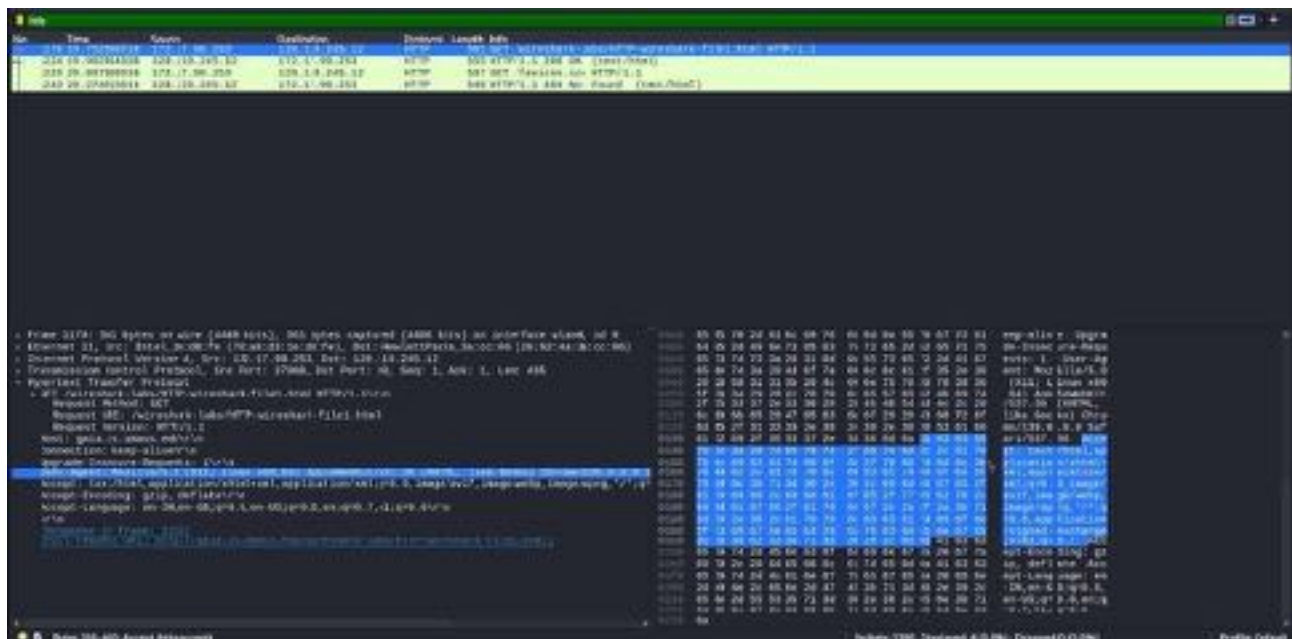
No.	Time	Source	Destination	Protocol	Length	Info
1178	17.70250410	172.17.0.253	172.17.0.253	HTTP	501	SET /wirespark-labs/HTTP-wirespark-file.html HTTP/1.1
1179	17.70250450	172.17.0.253	172.17.0.253	HTTP	555	HTTP/1.1 200 OK (text/html)
1229	26.00790016	172.17.0.253	172.17.0.253	HTTP	507	SET /favicon.ico HTTP/1.1
1243	26.27481964	172.17.0.253	172.17.0.253	HTTP	540	HTTP/1.1 404 NOT FOUND (text/html)

- Frame 1243: 540 bytes on wire (4320 bits), 540 bytes captured (4320 bits) on interface wlan0, id 0 - Ethernet II, Src: IntelE1000e (78:1d:63:2e:56:fe), Dst: IntelE1000e (78:1d:63:2e:56:fe) - Internet Protocol Version 4, Src: 172.17.0.253, Dst: 172.17.0.253 - Transmission Control Protocol, Src Port: 37688, Dst Port: 80, Seq: 1, Ack: 1, Len: 405 - Hypertext Transfer Protocol - HTTP/1.1 404 NOT FOUND - Response Version: HTTP/1.1 - Status Code: 404 - Reason (code description): Not Found - Content-Type: text/html; charset=utf-8 - Server: Apache/2.4.18 (Ubuntu) - Content-Length: 530 - Content-Type: text/html; charset=utf-8 - Connection: keep-alive - Content-Type: text/html; charset=utf-8 - Time taken to download: 0.2043119 seconds - Response (61): /wirespark-labs/HTTP-wirespark-file.html	<pre> 0000 8c 64 04 00 00 00 75 75 21 20 4f 70 85 0a 53 53 - 1 0010 40 2f 30 20 32 20 32 20 60 0f 64 1f 70 00 72 8c - 1 0020 64 1f 20 20 33 20 20 20 60 0f 64 1f 70 00 72 8c - 1 0030 14 0a 2f 20 20 20 60 0f 00 0a 12 00 10 0a 0f 12 - 1 0040 53 52 21 31 00 8a 80 81 11 10 20 60 6f 64 00 00 - 1 0050 80 80 40 20 70 8a 75 61 21 12 12 20 20 47 61 71 - 1 0060 30 10 10 00 20 20 31 3a 10 30 30 30 30 30 30 30 - 1 0070 60 00 00 05 5a 61 61 61 3a 20 22 30 30 20 30 30 - 1 0080 42 20 30 62 00 32 31 3f 6a 40 3f 12 6a 6a 61 61 - 1 0090 60 00 17 74 00 0f 61 6a 67 65 73 3a 20 51 78 74 - 1 0100 40 72 60 6a 62 6f 6a 74 61 8a 74 2a 6a 6a 6a 67 - 1 0110 74 60 30 20 31 32 30 60 6a 45 65 65 70 20 41 3c - 1 0120 60 70 62 20 21 0a 00 60 67 75 74 31 30 20 20 20 - 1 0130 60 65 71 30 31 30 30 60 6a 43 6f 6a 6a 65 61 74 - 1 0140 40 60 6a 6a 6a 6a 6a 6a 7a 6a 6a 6a 6a 6a 6a 6a - 1 0150 6a 43 6f 6a 74 65 6a 74 21 54 70 70 65 6a 50 74 - 1 0160 60 70 71 2f 60 71 60 61 20 20 60 65 72 73 65 - 1 0170 74 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a - 1 0180 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a - 1 0190 70 20 20 20 18 3f 70 27 71 65 20 84 6f 73 6a 6a - 1 0200 4f 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a 6a - 1 0210 6a 12 17 10 20 27 2f 87 61 69 62 2a 60 72 2a 70 - 1 0220 6a 61 71 71 20 65 6a 70 2f 73 68 72 65 71 60 61 - 1 </pre>
--	--

6. How many bytes of content are being returned to your browser?



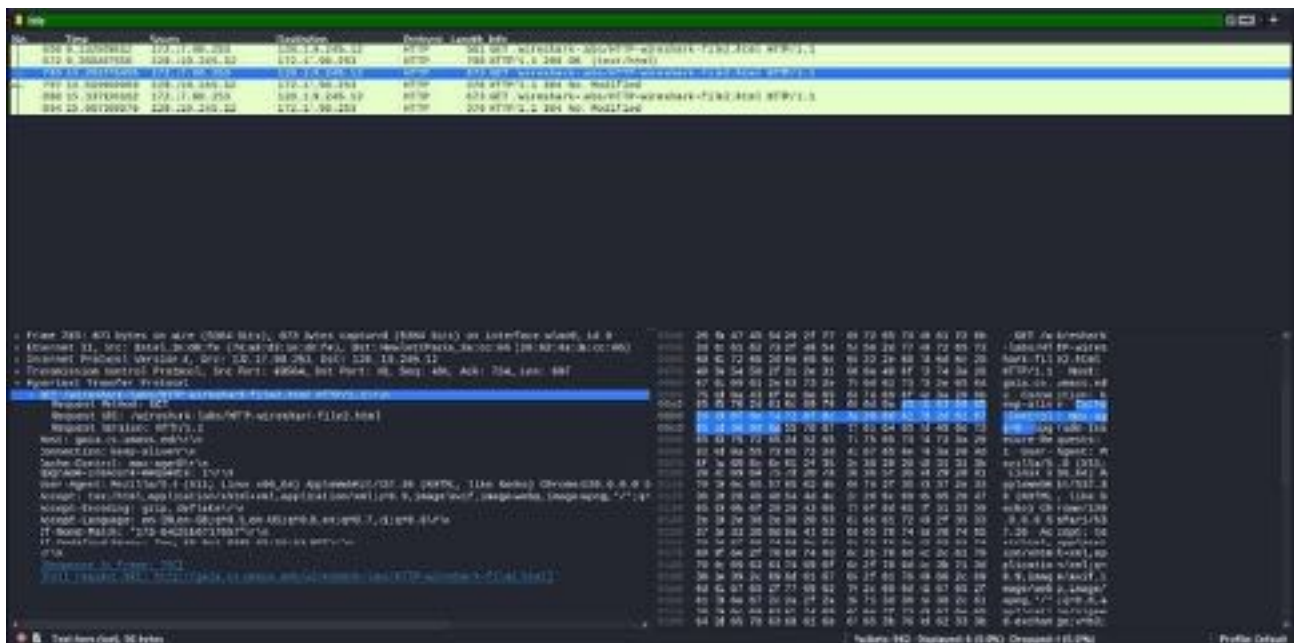
7. By inspecting the raw data do you see any headers within the data that are not displayed in the packet-listing window? If so, name one.



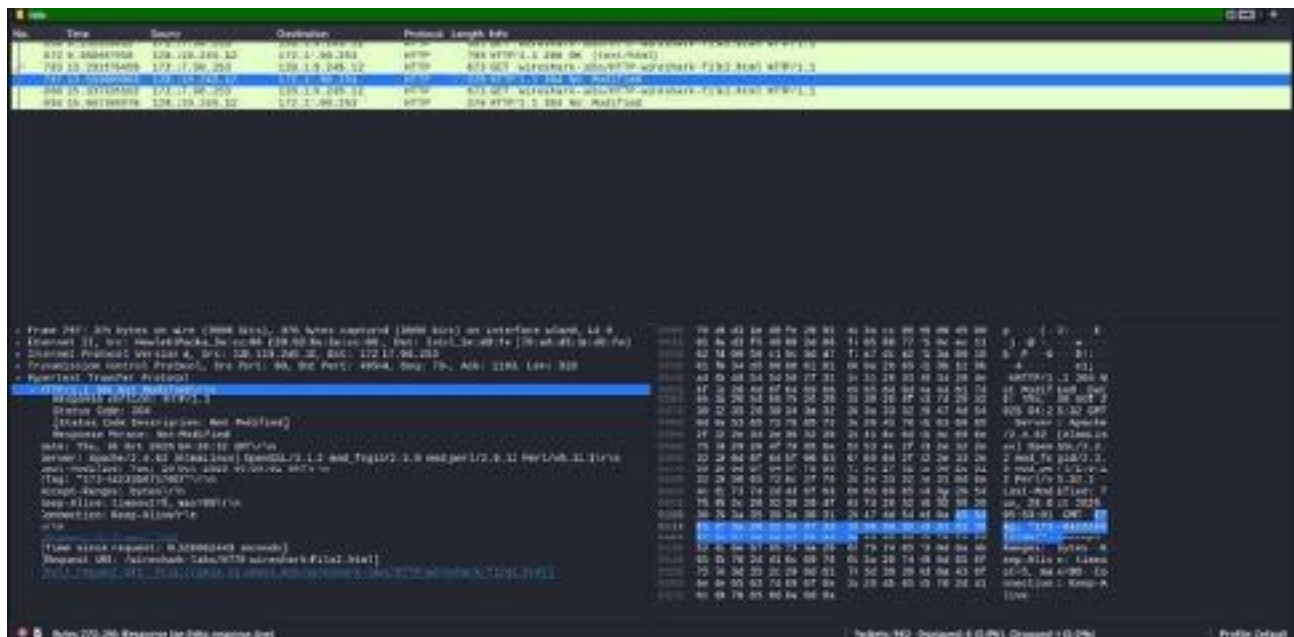
8. First HTTP GET... Do you see an "IF-MODIFIED-SINCE" line?

9. ...server response... Did the server explicitly return the contents of the file? How can you tell?

10. ...second HTTP GET... Do you see an "IF-MODIFIED-SINCE:" line? If so, what information follows?



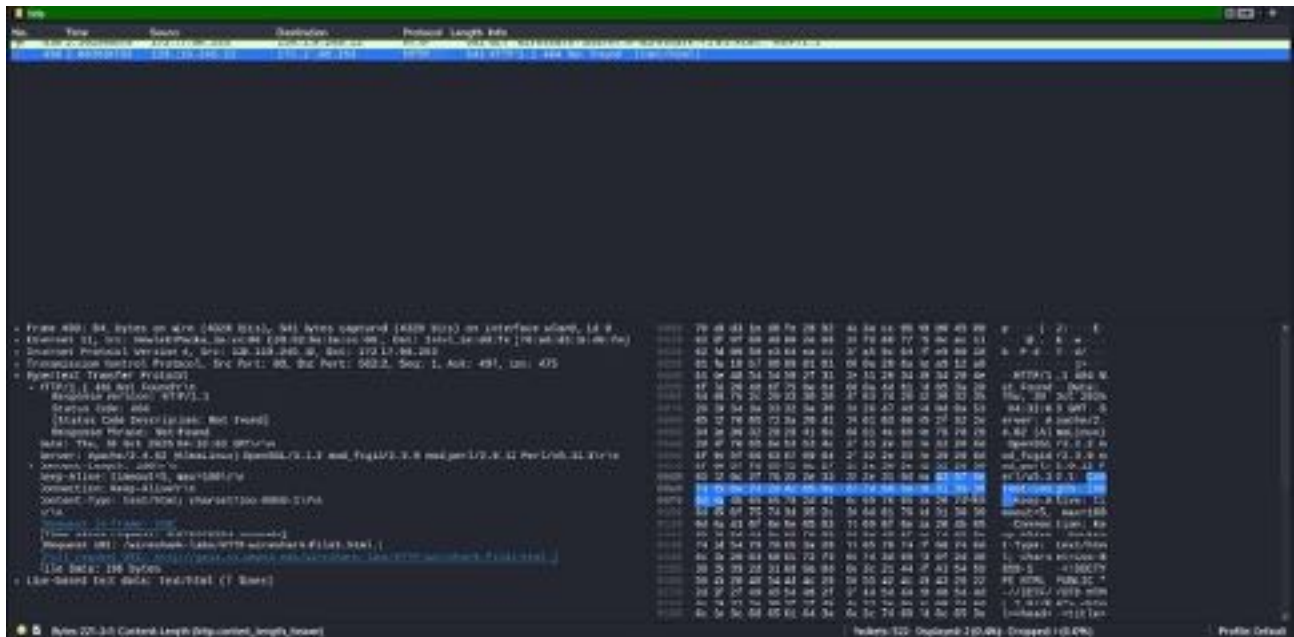
11. What is the HTTP status code... in response to this second HTTP GET? Did the server return the contents of the file?



TCP Segmentation: A single large HTTP response (like a big file) is too large for one packet. The Transport Layer (TCP) breaks the single response into multiple pieces, called **segments**. Each segment is sent in its own IP packet.

- **Wireshark "Continuation":** Wireshark is smart. It sees these multiple TCP packets all belonging to the *same* HTTP response. It labels the first packet as HTTP/1.1 200 OK and marks all the subsequent packets for that same file as **TCP segment of a reassembled PDU** or "Continuation". This is Wireshark's label, not a real HTTP message

12. How many HTTP GET request messages were sent by your browser?



Embedded Objects: A webpage (base HTML file) usually contains references to other objects (images, scripts) with their own URLs.

- **Multiple GETs:** The browser must download the base HTML file first, read it, and then issue *new* GET requests for *each* embedded object it finds.
- **Parallel Downloads:** Modern browsers open multiple TCP connections at once to download these objects in parallel (at the same time) to speed up page loading.

Setup Steps:

1. Clear browser cache.
2. Start Wireshark capture with httpfilter.
3. Browse to <http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file4.html>.
4. Stop capture.

16. How many HTTP GET request messages were sent by your browser? To which Internet addresses?

- **Step 1:** Look at your filtered packet list. You will see **three** GET requests.
 1. The first GET is for the base HTML file (...file4.html).
 2. The second GET is for the first image (e.g., .../pearson.gif).
 3. The third GET is for the second image (e.g., .../kurose_ross.gif).
- **Step 2:** To find the addresses, click each GET request and look at the Host: header in the Hypertext Transfer Protocol section, or the Destination: IP in the Internet Protocol section.

1. GET ...file4.html goes to gaia.cs.umass.edu.
2. GET .../pearson.gif goes to www.aw-bc.com.
3. GET .../kurose_ross.gif goes to manic.cs.umass.edu.

17. Can you tell whether your browser downloaded the two images serially, or in parallel? Explain.

- **Step 1:** Look at the **timestamps** in the "Time" column of your packet list.
- **Step 2:** Find the GET request for the first image (.../pearson.gif) and the GET request for the second image (.../kurose_ross.gif).
- **Step 3:** You will see that the GET for the second image is sent *before* the 200 OK response for the first image is received.
- **Answer:** They are downloaded in **parallel**. You can tell because the request for the second image is sent almost immediately after the request for the first, *without* waiting for the first image to finish downloading.

5. HTTP Authentication (Q18-19)

Theory for Report:

- **HTTP Basic Authentication:** A simple, *insecure* method for password-protecting a webpage.
- **The Process:**
 1. Browser sends a normal GET request.
 2. Server sees the page is protected and replies with a **401 Unauthorized** status. This response includes a **WWW-Authenticate: Basic ...** header, telling the browser it needs a password.
 3. The browser pops up a login box.
 4. The browser re-sends the GET request, but this time adds an **Authorization:** header.
- **Base64 Encoding:** The Authorization header contains the string "Basic" followed by the username and password joined by a colon (e.g., user:pass) and encoded in **Base64**. Base64 is *not* encryption; it's just a way to make text safe for transmission. It can be easily decoded by anyone.

Setup Steps:

1. Clear browser cache.
2. Start Wireshark capture with httpfilter.
3. Browse to
http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html.

4. A popup will appear. Enter username: wireshark-students and password: network.
5. Stop capture.

18. What is the server's response (status code and phrase) in response to the initial HTTP GET message?

- **Step 1:** Look at the first GET request and the packet immediately following it.
- **Step 2:** This response packet will have the **Status Code: 401** and **Status Phrase: Unauthorized**.
- **Step 3:** You can also expand Hypertext Transfer Protocol and see the WWW-Authenticate: header.

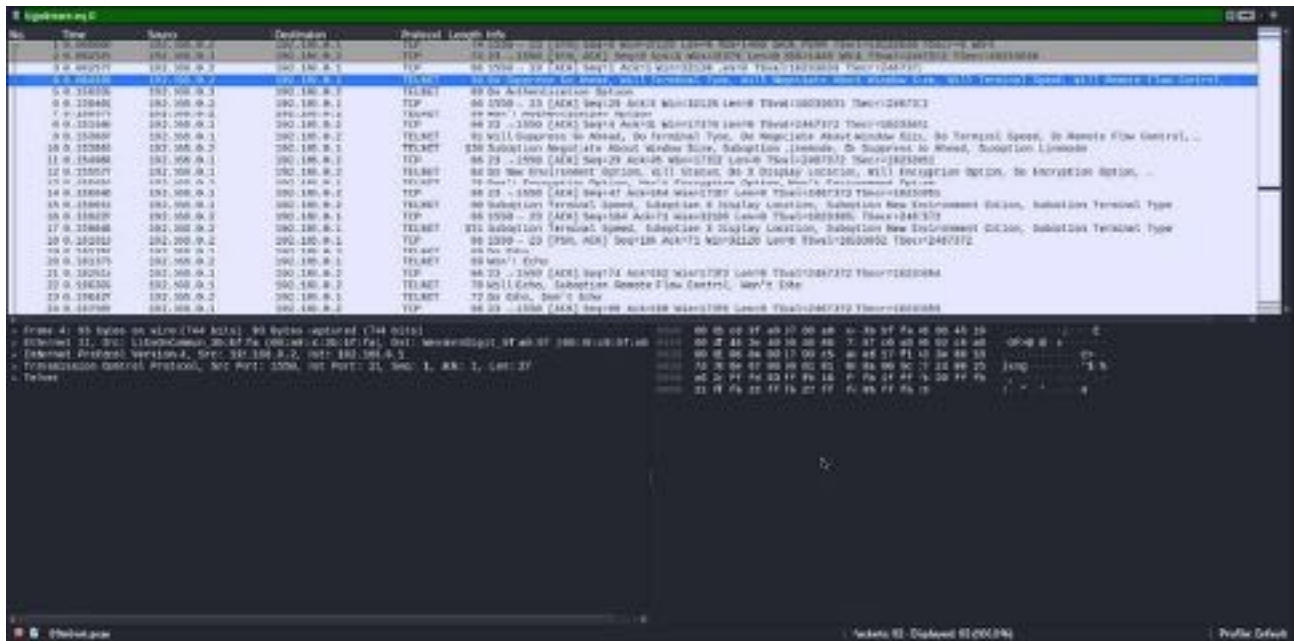
19. When your browser sends the HTTP GET message for the second time, what new field is included?

- **Step 1:** Click on the **second GET** request in your list (the one sent *after* the 401 response).
- **Step 2:** In "Packet Details," expand Hypertext Transfer Protocol.
- **Step 3:** You will see a new header: **Authorization:**.
- **Step 4:** If you expand it, you'll see Credentials: Basic d2lyZXNoYXJrLXN0dWRlbnRzOm5ldHdvcm0=, which is the Base64-encoded username and password.

Wireshark Ex 2025-26.pdf

- **Telnet** is an old and **insecure** remote login protocol. It transmits all data, including usernames and passwords, in **clear text**. Anyone sniffing the network can read everything.
- **TCP Stream:** Wireshark has a powerful feature that lets you reconstruct the full TCP conversation. By "Following" a TCP stream, you can see the complete, ordered flow of data between the client and server, making it easy to read protocols like Telnet or HTTP.

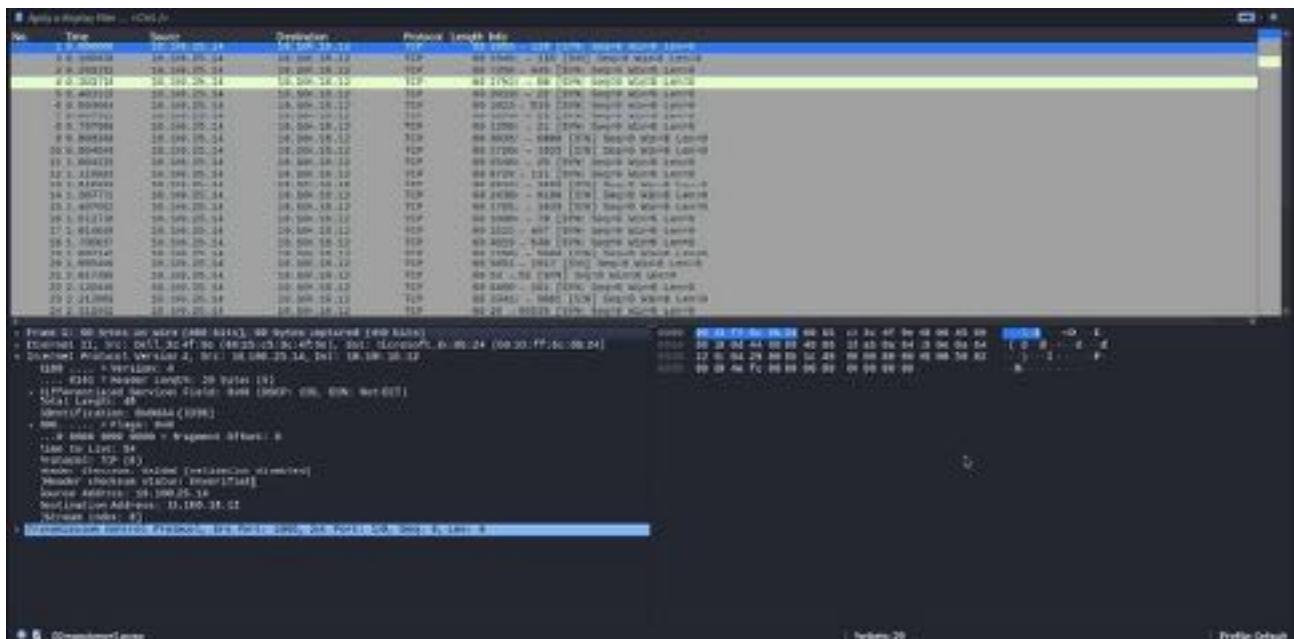
Q1: Who logged into 192.168.0.1? (Username & Password)



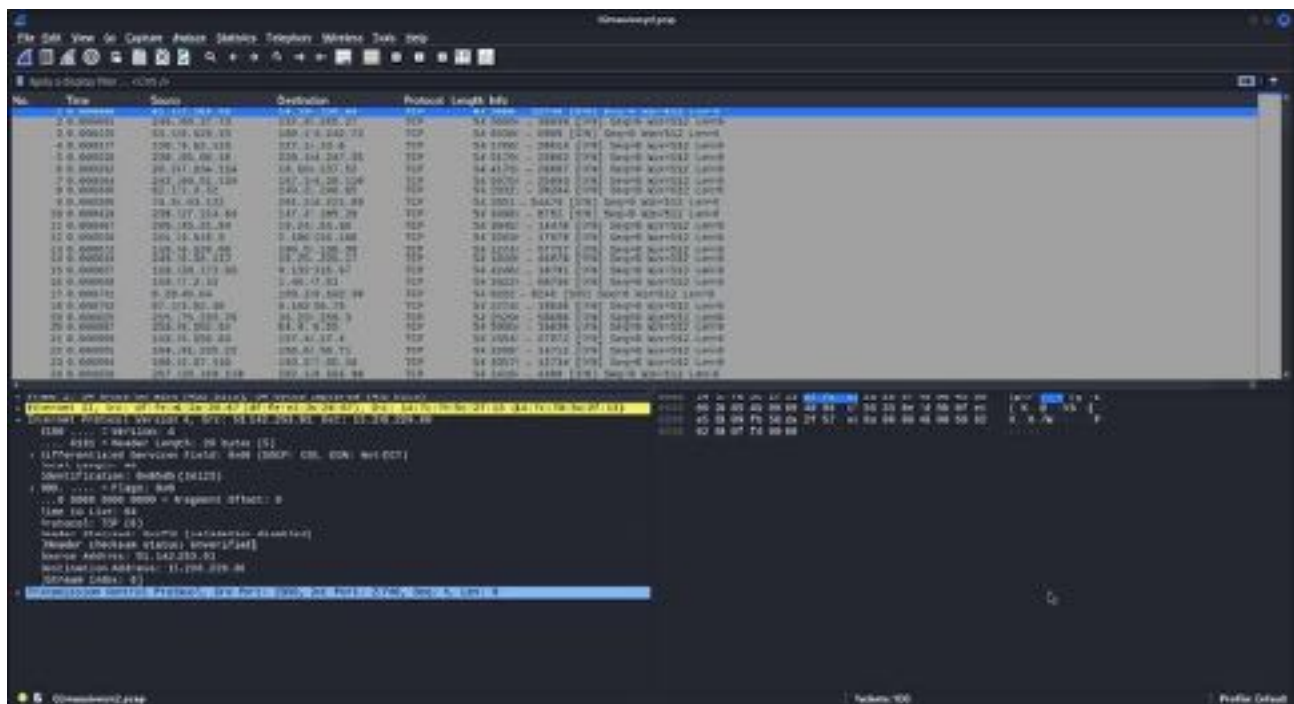
TCP3-Way Handshake: A normal TCP connection starts with a 3-way handshake:

1. Client sends [SYN]
 2. Server replies [SYN, ACK]
 3. Client replies [ACK]
- **SYN Flood Attack:** This is a "Denial of Service" (DoS) attack. The attacker sends a massive number of [SYN] packets to a server but **never** sends the final [ACK] packet. This forces the server to keep connections half-open, filling up its memory and resources until it can't accept any new, legitimate connections.
 - **IP Spoofing:** To make the attack harder to block, attackers often "spoof" (fake) the source IP address, making the [SYN] packets look like they are coming from many different random computers.

Q1: massivesyn1.pcapis an...



Q2: massivesyn2.pcapis an...

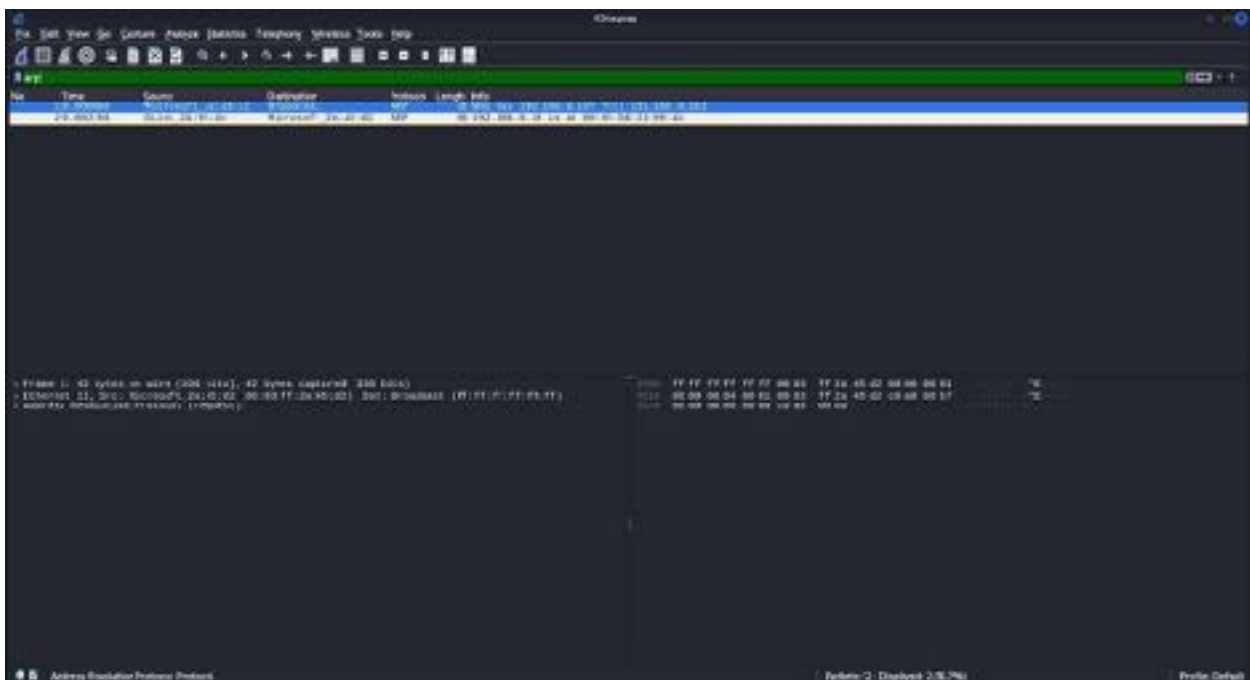
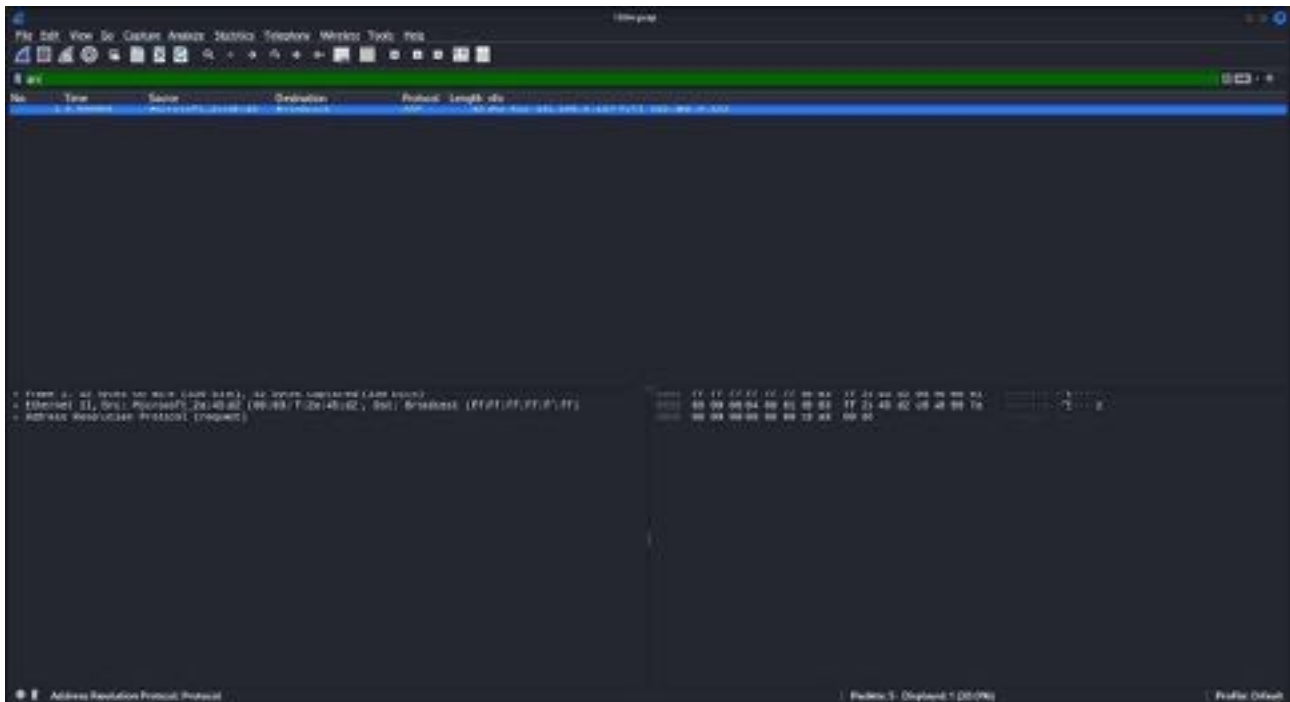


ARP (Address Resolution Protocol): When a computer (Jim) wants to send a packet to another computer (Risa) *on the same local network*, it knows Risa's IP address (e.g., 192.168.1.5) but doesn't know her physical **MAC address** (e.g., 0A:1B:2C:3D:4E:5F).

- ARP is used to fix this. Jim shouts (broadcasts) an **"ARP Request"** to everyone on the network: "Who has IP 192.168.1.5? Tell me (Jim's IP)."
- Risa's computer receives this, and only she sends back an **"ARP Reply"**: "I have 192.168.1.5. My MAC address is 0A:1B:2C:3D:4E:5F."

- If this process fails, Jim can't send any packets to Risa, and his machine might appear "offline."

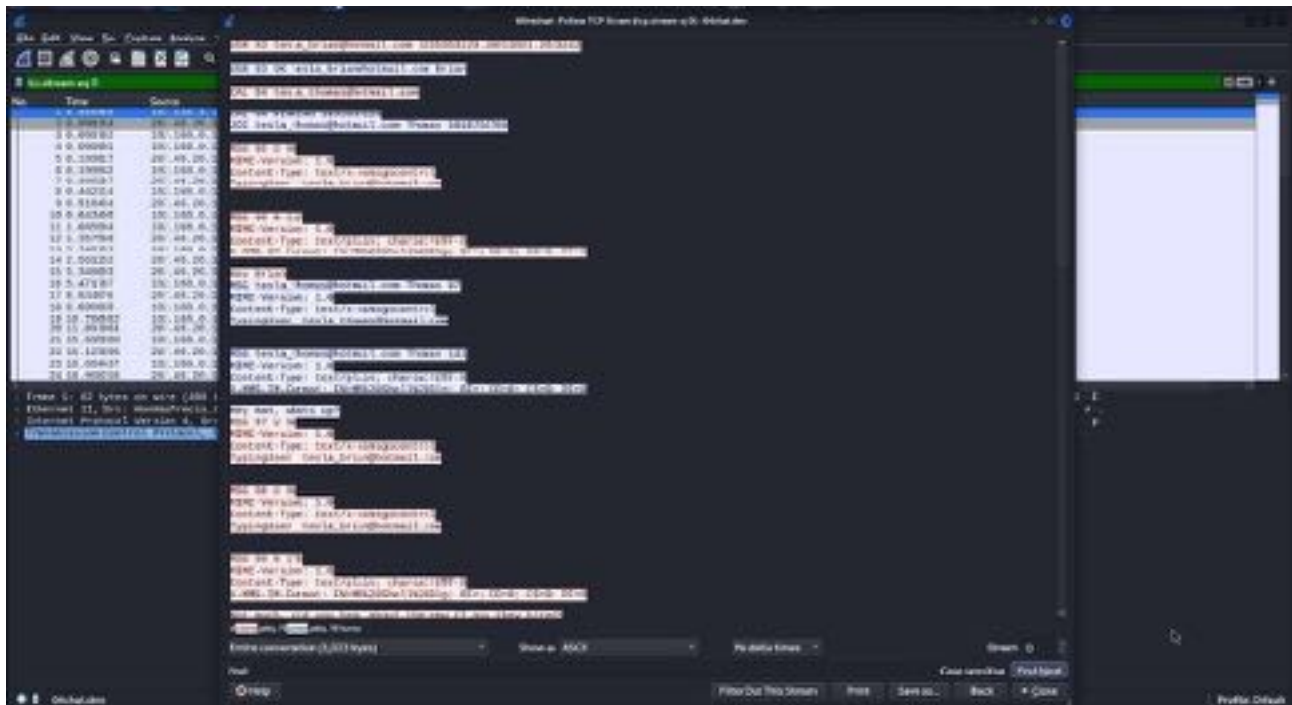
Q: Compare... and find out why Jim's machine is not online.



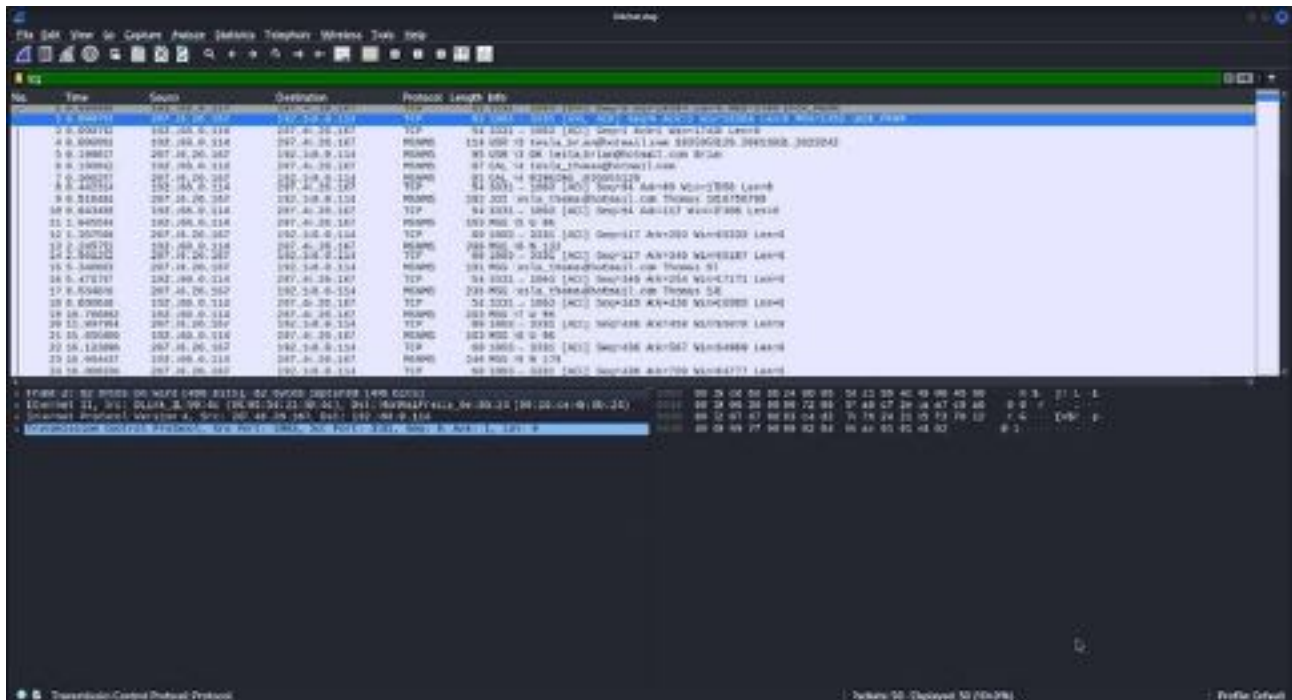
Plain-Text Protocols: Just like Telnet, many old chat protocols (like MSN Messenger, AIM, or IRC) sent all messages in clear text.

- **Protocol Analysis:** Wireshark is excellent at dissecting packets. Even if it doesn't explicitly label a protocol as "MSN," it can often identify it based on the port number or data. Following the TCP stream allows a network admin to reconstruct the entire conversation.

Q1: What kind of protocol is used?



Q2: This is the conversation between...



Q3: What do they say about you(sysadmin)?

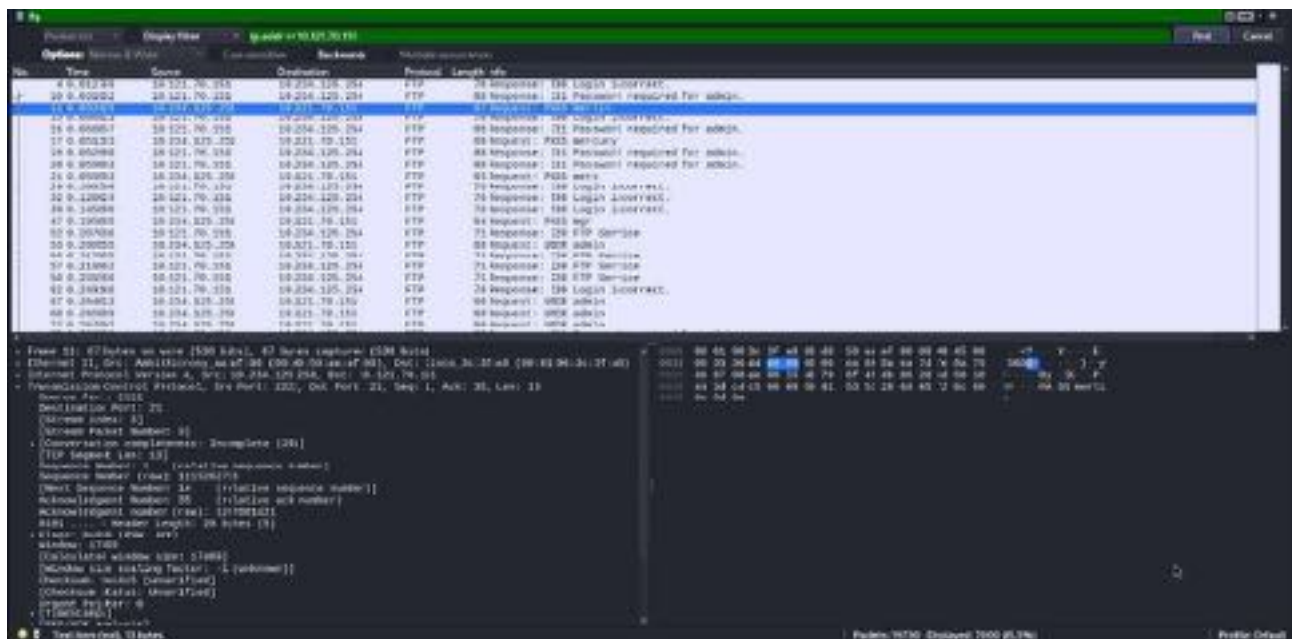


Wireshark Ex2 2025-26.pdf

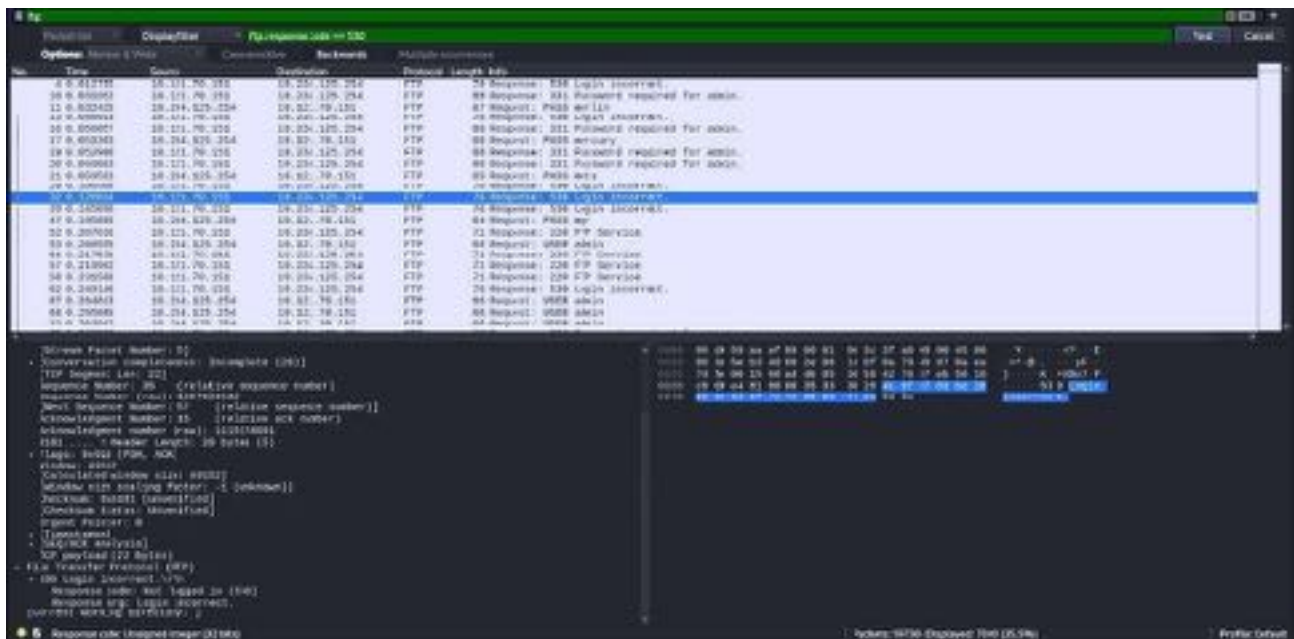
FTP (File Transfer Protocol): An outdated protocol for transferring files. It's notoriously insecure as it sends commands and data in clear text (just like Telnet).

- **FTP Control Connection:** FTP operates on **TCP port 21**. This connection is *only* for commands (like USER, PASS) and server responses (like 220 Welcome, 530 Login incorrect).
- **FTP Error Codes:** FTP uses 3-digit status codes. A 5xx code (e.g., 530) indicates a failed command or permanent error. **530 Login incorrect** is the standard response for a failed login attempt.
- **Brute-Force Attack:** A common attack where an attacker tries to guess a password by attempting to log in hundreds or thousands of times with different passwords. Seeing many 530 responses from one IP is a major red flag for this.

Q1: 10.121.70.151 is FTP...



Q2: 10.234.125.254 is FTP...



Investigating Unknown Traffic: When faced with a capture file containing unknown protocols, a good first step is to get a high-level summary of all the "conversations" (connections) happening.

- **Wireshark Conversations:** The Statistics -> Conversationstool is perfect for this. It lists every unique connection (IP-to-IP, Port-to-Port) and shows how many packets and bytes were exchanged. This quickly reveals which connections are the "chattiest" and what ports are being used.
- **IANA Port Numbers:** Most common protocols use "well-known ports" (0-1023), like HTTP (80) or FTP (21). Seeing high-volume traffic on an "ephemeral" port (above 1023) is suspicious and warrants investigation. Port 6346 is famously used by the Gnutella P2P file-sharing protocol.

Q1: See what's going on with Wireshark GUI

Wireshark packet capture showing a list of network packets. The interface includes a menu bar, toolbar, and packet details pane on the right. The packet list shows various protocols including TCP, UDP, and ICMP.

Q2: Which application uses TCP/6346? Check the web

Wireshark packet capture showing a list of network packets. The interface includes a menu bar, toolbar, and packet details pane on the right. The packet list shows various protocols including TCP, UDP, and ICMP.

Ex 7: Find out the total no. of TCP SYN packets for port 443.

Theory: We need to combine three conditions:

1. `tcp.port == 443`: Finds all packets (to or from) port 443 (HTTPS).
 2. `tcp.flags.syn == 1`: Checks if the SYN flag is "on".
 3. `tcp.flags.ack == 0`: (Optional but good practice) Ensures the ACK flag is "off", so you *only* get the initial [SYN] and not the [SYN, ACK] from the server.
- **Filter:** `tcp.port == 443 && tcp.flags.syn == 1 && tcp.flags.ack == 0`

- **How to find the total:** After applying the filter, the total number of packets is shown in the Wireshark status bar (Displayed: Y).

Ex 8: Filter out all the packets with the HTTP response code 200.

- **Theory:** Wireshark can parse HTTP and lets you filter on the response code field.
- **Filter:** `http.response.code == 200`

Ex 9: The attacker... <http://captive.apple.com/>. Write down the filter to identify the HTTP host.

- **Theory:** We filter on the Hostheader, which is part of every HTTP/1.1 request.
- **Filter:** `http.host == "captive.apple.com"`

Ex 10: Write down the filter to identify the destination port 23.

- **Theory:** Port 23 is for Telnet, which runs on TCP. We use `dstport` to specify *only* the destination.
- **Filter:** `tcp.dstport == 23`

Ex 11: Filter out the packets on behalf of the MAC address.

- **Theory:** We can filter at Layer 2 (Ethernet) using `eth.addr` to find a MAC address as either the source *or* destination.
- **Filter:** `eth.addr == 00:01:02:0a:0b:0c` (Replace with your target MAC address. The format can be `xx:xx:xx:xx:xx:xx` or `xx-xx-xx-xx-xx-xx`).

Ex 12: Write down the filter to identify the IP address 3.209.186.7 for port 443. Find out the total no. of packets.

- **Theory:** Combining an IP address (`ip.addr` for source *or* destination) and a port.
- **Filter:** `ip.addr == 3.209.186.7 && tcp.port == 443`
- **How to find the total:** The total number is in the Wireshark status bar (Displayed: Y).

BCSE319P LAB MANUAL-Ex1_.pdf

WHOIS / Registrar Lookup: Every domain name (like google.com) must be registered with a central authority. This registration data, called "WHOIS" information, is often public. It can include the name of the organization that registered the domain, the registration and expiration dates, and the **Name Servers (NS)** for that domain. The lab points to lookup.icann.org as a tool for this.

- **DNS Lookup:** As we saw in the previous labs, DNS (Domain Name System) translates names to IPs. Online tools like mxtoolbox.com or whatsmydns.net provide a user-friendly way to query a domain's DNS records from multiple locations around the world. This is useful for seeing how a website's DNS is structured

[illegible]

Registrar Information

Name: POP Ltd aka Public Domain Registry Ltd

CANA ID: 300

about:contact email: info@publicdomainregistry.com

about:contact phone: +44 (0)1223 335527

DNSSEC Information

Delegation Signed: signed

Delegation Signer Data:

Key Alg: 2551

Algorithm: 13

Unsigned: 0

Digest: 2870B5C0D51519B5CFE9A74EED16E8D1EE39DA352814C38F2233E47D505D24C2

Authoritative Servers

Registry Server URL: <https://dns.publicdomainregistry.net/AD/lookup/12.12.12>

Last updated from Registry: RDAF IDB: 2022-12-30T07:27:55Z

Registrar Server URL: <https://api.publicdomainregistry.com/api/1.0/lookup>

Last updated from Registrar: RDAF IDB: 2022-12-30-07:27:55Z

Notices and Remarks

Notices:

Data Policy:

The contact information for one or more entries associated with this domain has been redacted in accordance with the entry's disclosure preferences.

Terms of Use:

URL for Terms of Use: <https://www.publicdomainregistry.com/terms-of-use>

<https://dns.publicdomainregistry.net/AD/lookup>

Website Details:

For more information on domain policies and our platform visit: <https://www.brg.io>

<https://www.brg.io>

RDDoS Inaccuracy Complaint Form:

URL of the RDDoS Inaccuracy Complaint Form: <https://www.brg.io/rddos>

<https://www.brg.io/rddos>

Metadata

Registry Expires On: 2026-10-10 00:00:00 UTC

Updated On: 2025-10-10 05:24:11 UTC

Created On: 2025-10-11 04:30:00 UTC

Contact Information

Administrative:

Name: Nishu Khos

Organization: HOUSING DEVELOPMENT FINANCE CORPORATION LIMITED

Email: nishu@hdfc.com

Website: www.hdfc.com

Phone: tel:+91.56316467

Fax: tel:+91.20.46758

Kind: individual

Mailing Address: Churchgate, 169 Service House, Backbay Reclamation, Mumbai, Maharashtra, 40013

ISO 3166 Code: IN

Technical:

Name: Nishu Khos

Organization: HOUSING DEVELOPMENT FINANCE CORPORATION LIMITED

Email: nishu@hdfc.com

Website: www.hdfc.com

Phone: tel:+91.56316467

Fax: tel:+91.20.46758

Kind: individual

Mailing Address: Churchgate, 169 Service House, Backbay Reclamation, Mumbai, Maharashtra, 40013

ISO 3166 Code: IN

Registrant:

Name: Nishu Khos

Organization: HOUSING DEVELOPMENT FINANCE CORPORATION LIMITED

Email: nishu@hdfc.com

Website: www.hdfc.com

Phone: tel:+91.56316467

Fax: tel:+91.20.46758

Kind: individual

Exercise 1: Using any specific tool, find out the Certificate Issuer for vit.ac.in, the validity period, and the encryption algorithm used...

Theory for Report:

- Websites use **SSL/TLS certificates** to enable secure **HTTPS** connections. This certificate proves the website's identity (like a digital passport) and enables encryption.
- The certificate is "issued" by a trusted **Certificate Authority (CA)** (e.g., DigiCert, Let's Encrypt).
- Certificates have a **validity period** (a "not before" and "not after" date).
- They specify the **encryption algorithm** (e.g., RSA, SHA-256) used to secure the connection.

Steps

1. Open Firefox on your Kali machine.
2. Go to <https://vit.ac.in>.
3. Look to the left of the address bar. You will see a **padlock icon**. Click it.
4. Click on **"Connection secure"** -> **"More information"**.
5. A "Page Info" window will pop up. Click the **"View Certificate"** button.
6. A new tab will open showing the certificate details:
 - **Certificate Issuer:** Look in the "Issued By" section under "Common Name" (e.g., GoDaddy Secure Certificate Authority).
 - **Validity Period:** Look in the "Validity" section for the "Not Before" and "Not After" dates.
 - **Encryption Algorithm:** Look in the "Public Key Info" section. You'll see "Algorithm" (e.g., RSA) and "Key Size" (e.g., 2048 bit).

Exercise 2: Using any specific tool, find out the Server Type, the status, and the HTTP Delay Check.

Theory for Report:

- **Server Type:** When an HTTP server responds, it often includes a Server header that identifies the software it's running (e.g., Apache, nginx, Microsoft-IIS). This is valuable information for an attacker.
- **HTTPStatus:** The status code (like 200 OK) tells the client the result of its request.
- **HTTPDelay Check:** This measures the "Time to First Byte" (TTFB) — how long it takes for the server to send back the *first piece* of a response after receiving a request.

Steps

1. We can get all this information with one command: **curl**.
2. Open your Kali terminal.

3. Type the following command (all one line):

Bash

```
curl -svo /dev/null https://vit.ac.in/
```

- **-s:** Silent mode (don't show progress bar).
 - **-v:** Verbose (show all the connection details and headers).
 - **-o /dev/null:** Dumps the actual page content so it doesn't fill your screen.
 - **https://vit.ac.in/:** The target URL.
4. Press Enter. You will see a lot of text. Look for these lines (they will be mixed in with other data):
 - **Status:** You'll see a line starting with < HTTP/2. It will show the status code, e.g., < HTTP/2 200. (This means 200 OK).
 - **Server Type:** You'll see a line starting with < server:. The value after this is the server type (e.g., < server: Apache).
 - **HTTP Delay Check (TTFB):** You'll see timings for the whole connection. Look for TTFB:. The time next to this (e.g., 0.215487s) is your delay check. (If curl doesn't show TTFB, the starttransfer time is a good equivalent).

Exercise 3: Using any specific tool, ping the IP Address (e.g. 142.250.182.3) Theory for

Report:

- **ping** is the most basic network-testing tool. As we saw in the last lab, it uses **ICMP Echo Requests** to see if a host is "alive" and to measure the **Round Trip Time (RTT)** — how long it takes for a packet to get to the host and back.

Steps

1. Open your Kali terminal.
2. Type the ping command followed by the IP address. The **-c 4** flag tells it to send only 4 packets (like Windows) instead of pinging forever.

Bash

```
ping -c 4 142.250.182.3
```

3. Press Enter.
4. You will see the replies from the IP address, showing the RTT in milliseconds (ms). The summary at the end shows you if any packets were lost.

What is MSRPC port 135?

- **Answer:** Port 135 (TCP/UDP) is used for the Microsoft Remote Procedure Call (MSRPC) service. It's a fundamental service in Windows that allows different processes to communicate with each other, both on the same machine and across a network. It's essentially the "receptionist" for Windows services, telling a client program what port a specific service (like Exchange Server) is running on.

What is port 135 exploit?

- **Answer:** Because MSRPC is so deeply integrated into Windows, it has had many vulnerabilities. A "port 135 exploit" usually refers to an attack that targets one of these vulnerabilities. The most famous was the Blaster Worm (2003), which used a buffer overflow vulnerability in the MSRPC service to infect millions of computers without any user action.

How will you close MSRPC port 135?

- **Answer:** You should not close port 135 on a Windows machine, as it's critical for the OS and many applications to function. The correct way to secure it is to:
 1. **Use a Firewall:** Place a firewall (like the built-in Windows Firewall or a network firewall) to block all *external* access to port 135 from the internet.
 2. **Keep Windows Updated:** Apply all security patches from Microsoft to fix any new vulnerabilities found in the MSRPC service.

What is Port 999 used for?

- **Answer:** Port 999 is not officially assigned to a single, well-known service by IANA (the internet standards body). However, it is commonly used by:
 - **Password-protected telnet:** An unofficial alternative to the standard port 23.
 - **Application-specific services:** Many custom applications use it (e.g., some games, remote admin tools).
 - **Malware:** Because it's often open, some trojans or backdoors may use this port to communicate.

Nmap Scan Commands

Command 1: Scanning a Single IP Command: nmap

172.105.33.179

This is a basic Nmap scan. By default, it first sends a ping (an ICMP echo request) to see if the host is online. If it is, Nmap then scans the 1,000 most common TCP ports (like port 80 for HTTP, 443 for HTTPS, 22 for SSH, etc.) to see which ones are open.

```
(kali@kali)~$ nmap 172.105.33.179
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 13:44 IST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.12 seconds

(kali@kali)~$
```

Command 2: Scanning a Localhost

- Command: nmap 127.0.0.1
- 127.0.0.1 is the loopback address, which always points to your own machine. This scan is used to check for any network services you are running on your own Kali machine.

```
(kali@kali)~$ nmap 127.0.0.1
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 13:45 IST
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000010s latency).
All 1000 scanned ports on localhost (127.0.0.1) are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 1 IP address (1 host up) scanned in 0.10 seconds

(kali@kali)~$
```

Command 3: Scanning a Host

- **Command:** `nmap www.vit.ac.in`
- Nmap can accept hostnames as targets. It will first perform a DNS lookup to find the IP address for `www.vit.ac.in` and then scan that IP.

```
(kali@kali)-[~]
└─$ nmap www.vit.ac.in
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 13:46 IST
Nmap scan report for www.vit.ac.in (122.184.65.22)
Host is up (0.0027s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http
113/tcp   closed ident
443/tcp   open  https
8080/tcp  closed http-proxy

Nmap done: 1 IP address (1 host up) scanned in 5.39 seconds
```

Command 4: Scanning Multiple & Range of IPs

- **Commands:**
 - `nmap 192.168.0.1 192.168.0.2 192.168.0.3` (Scans 3 specific IPs)
 - `nmap 192.168.0.*` (Scans the entire /24 subnet, from 192.168.0.1 to 192.168.0.254)
 - `nmap 172.105.33.179-200` (Scans the 22 IP addresses in that range)
- This demonstrates Nmap's flexible target selection, allowing pentesters to scan multiple hosts or entire subnets with a single command.

```
(kali@kali)-[~]
└─$ nmap 192.168.0.1 192.168.0.2 192.168.0.3
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 13:47 IST
Nmap done: 3 IP addresses (0 hosts up) scanned in 5.12 seconds
```

```
(kali@kali)-[~]
└─$ nmap 192.168.0.*
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 13:47 IST
Nmap done: 256 IP addresses (0 hosts up) scanned in 206.34 seconds
```

```

kali@kali: ~$
kali@kali: ~$ nmap 172.165.0.179 200
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-10 13:51:15+
Nmap scan report for 172-165-33-180.ip.linodeusercontent.com (172.165.33.180)
Host is up (0.029s latency).
Not shown: 995 filtered tcp ports (no-response)
PORT      STATE SERVICE
8081/tcp   closed http
443/tcp    open  https
8080/tcp    closed http proxy
8080/tcp    open  unknown
8080/tcp    open  sunwebadmin

```

```

Nmap scan report for 172.165.0.179.ip.linodeusercontent.com (172.165.33.180)
Host is up (0.009s latency).
Not shown: 997 filtered tcp ports (no response)
PORT      STATE SERVICE
80/tcp     open  http
8080/tcp    open  https
8080/tcp    open  ssh-xxx

```

```

Nmap scan report for 172-165-33-182.ip.linodeusercontent.com (172.165.33.182)
Host is up (0.031s latency).
Not shown: 985 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp     open  ftp
22/tcp     open  ssh
25/tcp     open  smtp
53/tcp     open  domain
80/tcp     open  http
110/tcp    open  pop3
111/tcp    open  pop3mid
143/tcp    open  imaps
443/tcp    open  https
465/tcp    open  smtps
587/tcp    open  submission
993/tcp    open  imaps
995/tcp    open  pop3s
3306/tcp   open  mysql

```

```

Nmap scan report for infraresolver-4.muni.in.prod.linode.com (172.165.33.183)
Host is up (0.030s latency).
All 1000 scanned ports on infraresolver-4.muni.in.prod.linode.com (172.165.33.183) were in ignored states.
Not shown: 999 filtered tcp ports (no-response), 21 filtered tcp ports (no-response)

```

```

Nmap scan report for 172-165-33-186.ip.linodeusercontent.com (172.165.33.186)
Host is up (0.031s latency).
Not shown: 995 closed tcp ports (reset)
PORT      STATE SERVICE
8081/tcp   open  http
443/tcp    open  https
587/tcp    open  submission
8080/tcp    open  nginxhttp2s
2222/tcp   open  FfcrNetTP-1

```

```

Nmap scan report for 172.165.0.179.ip.linodeusercontent.com (172.165.33.180)
Host is up (0.009s latency).
Not shown: 996 filtered tcp ports (no response)
PORT      STATE SERVICE

```

Command 2: Scanning Multiple IPs

```

# Command 1

```

```

# Command 2

```

```

# Command 3

```

```

# Command 4

```

```

# Command 5

```

```

# The command will scan a range of
# IP addresses and return a single
# report for each IP.

```

```

# Command 6

```

```

# Command 7

```

```

# Command 8

```

```

# Command 9

```

```

# Command 10

```

```

# Command 11

```

```

# Command 12

```

```

# Command 13

```

```

# Command 14

```

```

# Command 15

```

```

# Command 16

```

```

# Command 17

```

```

# Command 18

```

```

# Command 19

```

```

# Command 20

```

```

# Command 21

```

```

# Command 22

```

```

# Command 23

```

```

# Command 24

```

```

# Command 25

```

```

# Command 26

```

```

# Command 27

```

```

# Command 28

```

```

# Command 29

```

```

# Command 30

```

```

# Command 31

```

```

# Command 32

```

Not shown: 998 closed tcp ports (reset)

PORT	STATE	SERVICE
22/tcp	open	ssh
9999/tcp	open	unknown

Nmap scan report for 172-105-33-196.ip.linodeusercontent.com (172.105.33.196)

Host is up (0.022s latency).

Not shown: 995 closed tcp ports (reset)

PORT	STATE	SERVICE
22/tcp	open	ssh
80/tcp	open	http
443/tcp	open	https
3000/tcp	open	php
8443/tcp	open	https-alt

Nmap scan report for inpro2.fromot.com (172.105.33.197)

Host is up (0.012s latency).

Not shown: 887 filtered tcp ports (no-response), 99 closed tcp ports (reset)

PORT	STATE	SERVICE
21/tcp	open	ftp
22/tcp	open	ssh
25/tcp	open	smtp
53/tcp	open	domain
80/tcp	open	http
110/tcp	open	pop3
143/tcp	open	imap
443/tcp	open	https
465/tcp	open	smtps
587/tcp	open	submission
993/tcp	open	imaps
995/tcp	open	pop3s
2525/tcp	open	ms v worlds
3306/tcp	open	mysql

Nmap scan report for 172-105-33-198.ip.linodeusercontent.com (172.105.33.198)

Host is up (0.029s latency).

Not shown: 999 closed tcp ports (reset)

PORT	STATE	SERVICE
9100/tcp	open	jetdirect

Nmap scan report for 172-105-33-199.ip.linodeusercontent.com (172.105.33.199)

Host is up (0.029s latency).

Not shown: 997 filtered tcp ports (no-response)

PORT	STATE	SERVICE
22/tcp	open	ssh
80/tcp	open	http
443/tcp	open	https

Nmap scan report for 172-105-33-200.ip.linodeusercontent.com (172.105.33.200)

Host is up (0.031s latency).

Not shown: 996 closed tcp ports (reset)

PORT	STATE	SERVICE
22/tcp	open	ssh
80/tcp	open	http
443/tcp	open	https
3000/tcp	open	php

Nmap done: 22 IP addresses (19 hosts up) scanned in 17.20 seconds

Command 5: Excluding Hosts

- Command: `nmap 192.168.0.* --exclude 192.168.0.2`
- The `--exclude` flag is used to skip scanning a specific host. This is useful if you want to scan an entire network but avoid scanning a sensitive server (like a domain controller) or your own machine.

```
(kali@kali)~$ nmap 192.168.0.* --exclude 192.168.0.2
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 13:53 IST
Nmap done: 255 IP addresses (0 hosts up) scanned in 705.15 seconds
```

Command 6: OS Information & Ping Scan

- Commands:
 - `nmap -A 192.168.0.1`
 - `nmap -O 192.168.0.1`
 - `nmap -sn 192.168.1.1`
- Theory
 - **-A (Aggressive Scan):** This is a powerful "all-in-one" scan. It enables OS Detection (`-O`), Version Detection (`-sV`), Script Scanning (`-sC`), and Traceroute (`--traceroute`) all at once.
 - **-O (OS Detection):** This scan sends a series of probes and analyzes the TCP/IP stack responses to make an educated guess about the target's Operating System (e.g., "Windows 10", "Linux 5.4 kernel").
 - **-sn (Ping Scan):** This is a "no-ports" scan. It only pings the target to see if it's online and does *not* scan any ports. This is useful for quickly discovering all live hosts on a network.

```
(kali@kali)~$ nmap -A 192.168.0.1
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 13:57 IST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.97 seconds
```

```
(kali@kali)~$ nmap -O 192.168.0.1
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 13:58 IST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.15 seconds
```

```
(kali@kali) [~]
$ nmap -sn 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 13:58 IST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 1.10 seconds
```

Command 7: TCP SYN Scan (Stealth Scan)

- Command: `nmap -sS 192.168.1.1`
- This is the default scan type when run as root/sudo . It's called a "stealth scan" because it never completes the 3-way handshake.
 - It sends a SYNpacket.
 - If it gets a SYN/ACKback, the port is open. Nmap sends RSTto tear down the connection.
 - If it gets an RSTback, the port is closed.
 - This is fast and less likely to be logged by old firewalls. This requires root/sudo privileges

```
(kali@kali) [~]
$ nmap -sS 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 13:59 IST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.10 seconds
```

Command 8: TCP Connect Scan

- Command: `nmap -sT 192.168.1.1`
- This is the default scan type when a user does *not* have root privileges. It uses the operating system's connect system call to perform a full 3-way handshake. It's "noisier" (easier to detect and log) and slower than a SYN scan

```
(kali@kali) [~]
$ nmap -sT 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:00 IST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.11 seconds
```

Command 9: ACK Scan

- **Command:** `nmap -sA 192.168.1.1`
- This scan's purpose is not to find open ports. Its purpose is to map firewall rulesets. It sends an ACK packet.
 - If it gets an RSTback, the port is marked unfiltered(the firewall allowed the packet through).
 - If it gets no response (or an ICMP error), the port is marked filtered(the firewall blocked the packet)

```
(kali@kali)-[~]  
$ nmap -sA 192.168.1.1  
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:00 IST  
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn  
Nmap done: 1 IP address (0 hosts up) scanned in 3.12 seconds
```

Command 10: UDP Scan

- **Command:** `nmap -sU 192.168.1.1`
- This scan looks for open UDP ports. UDP is connectionless, making it much harder and slower to scan than TCP. This is important for finding services like DNS (port 53) or SNMP (port 161)

```
(kali@kali)-[~]  
$ nmap -sU 192.168.1.1  
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:01 IST  
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn  
Nmap done: 1 IP address (0 hosts up) scanned in 3.10 seconds
```

Command 11: Other Scan Types & Options

- `nmap -PR 192.168.1.1`: Performs an ARP scan on the local network. This is the fastest way to find live hosts on your LAN.
- `nmap -n 192.168.1.1`: The `-n` flag tells Nmap not to perform DNS resolution. This can speed up scans significantly if you are scanning many IPs.
- `nmap -p 1-30 192.168.1.1`: Scans a specific port range (1-30).
- `nmap -p- 192.168.1.1`: The `-p-` flag scans all 65,535 TCP ports.
- `nmap -F 192.168.1.1`: A Fast scan that scans only the 100 most common ports (instead of 1000).

- **nmap -sV 192.168.1.1:** Service Version detection. This is extremely useful. It connects to open ports and grabs the "banner" to determine the exact software and version running (e.g., Apache httpd 2.4.41, OpenSSH 8.2p1).
- **nmap -T0 ... -T3 ... :** Timing Templates. -T0 (Paranoid) is extremely slow and used to evade detection. -T4(Aggressive) is very fast. -T3(Normal) is the default.

```
(kali@kali)-[~]
$ nmap -PR 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:01 IST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.08 seconds
```

```
(kali@kali)-[~]
$ nmap -n 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:02 IST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.11 seconds
```

```
(kali@kali)-[~]
$ nmap -p 1-65535 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:02 IST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.20 seconds
```

```
(kali@kali)-[~]
$ nmap -p- 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:03 IST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.08 seconds
```

```
(kali@kali) [~]
$ nmap -F 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:03 IST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.08 seconds
```

```
(kali@kali) [~]
$ nmap -sV 192.168.1.1
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:04 IST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.14 seconds
```

```
(kali@kali) [~]
$ nmap -iU ... ..
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:04 IST
Failed to resolve "...".
Failed to resolve "...".
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.07 seconds
```

Nmap Scripting Engine (NSE)

Command 12: Vulnerability Scan

- Command: `nmap --script vuln rediff.com`
- The Nmap Scripting Engine (NSE) uses Lua scripts to automate tasks. The `--script vuln` command runs all scripts in the "vulnerability" category to actively check the target for known exploits and security holes.

```
(kali@kali) [~]
$ nmap --script vuln rediff.com
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:06 IST
Pre-scan script results:
| Broadcast-avahi-dos:
|   Discovered hosts:
|     224.0.0.251
|   After MITM avahi packet DOS (CVE-2011-1002).
|_ Hosts are all up (not vulnerable).
```

Command 13: Bug Bounty Scripts

- **Commands:**

- **nmap --script http-vuln-cve2017-5638 br-icloud.com.br:** Checks for a specific Remote Code Execution vulnerability in Apache Struts.
- **nmap -sv -p 443 --script ssl-enum-ciphers 15.206.152.201:** Checks an SSL/TLS server to see which encryption ciphers it supports, looking for weak or outdated ones.
- **nmap --script smb-vuln-* 89.117.188.67 :** Checks a host for all known SMB (Windows file sharing) vulnerabilities, such as EternalBlue.
- **nmap --script=smtplib-vuln-cve2011-1764 -pT:25,465,587 <host>:** Checks mail servers for a specific vulnerability.
- **nmap --script mysql-enum vit.ac.in:** Tries to enumerate (list) information from a MySQL database server.

```
(kali@kali)-[~]
$ nmap --script http-vuln-cve2017-5638 br-icloud.com.br
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:08:13
Nmap scan report for br-icloud.com.br (103.224.182.242)
Host is up (0.28s latency).
rDNS record for 103.224.182.242: 1h 182 242.above.com
Not shown: 499 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http
443/tcp    open  https

Nmap done: 1 IP address (1 host up) scanned in 38.15 seconds
```

```

(kali@kali) [~]
$ nmap -sv -p 443 --script ssl-enum-ciphers 15.206.152.201
Nmap 7.95 ( https://nmap.org )
Usage: nmap [Scan Type(s)] [Options] {target specification}
TARGET SPECIFICATION:
  Can pass hostnames, IP addresses, networks, etc.
  Ex: scanme.nmap.org, microsoft.com/24, 192.168.0.1; 10.0.0-255.1-254
  -iL <inputfilename>: Input from list of hosts/networks
  -iR <num hosts>: Choose random targets
  --exclude <host1[,host2][,host3],...>: exclude hosts/networks
  --excludefile <exclude_file>: Exclude list from file
HOST DISCOVERY:
  -sL: List Scan - simply list targets to scan
  -sn: Ping Scan - disable port scan
  -Pi: Treat all hosts as online -- skip host discovery
  PS/PA/PY[portlist]: TCP SYN, TCP ACK, UDP or SCTP discovery to given ports
  -PE/PP/PM: ICMP echo, timestamp, and netmask request discovery probes
  PU[protocol list]: IP Protocol Ping
  -n/-R: Never do DNS resolution/Always resolve [default: sometimes]
  --dns-servers <serv1[,serv2],...>: Specify custom DNS servers
  --system-dns: Use OS's DNS resolver
  --traceroute: Trace hop path to each host
SCAN TECHNIQUES:
  -sS/-sT/-sA/-sW/-sM: TCP SYN/Connect()/ACK/Window/Maimon scans
  -sU: UDP Scan
  -sM/-sF/-sX: TCP Muli, FIN, and Xmas scans
  --scanflags <flags>: Customize TCP scan flags
  -SI <zombie host[:probeport]>: Idle scan
  -sY/-sZ: SCTP INIT/COOKIE-ECHO scans
  -sO: IP protocol scan
  -b <IP relay host>: IP bounce scan
PORT SPECIFICATION AND SCAN ORDER:
  -p <port ranges>: Only scan specified ports
  Ex: -p22; -p1-65535; -p U:53,T:21-25,I:80,139,8080,S:9
  --exclude-ports <port ranges>: Exclude the specified ports from scanning
  -F: Fast mode - Scan fewer ports than the default scan
  -r: Scan ports sequentially - don't randomize
  --top-ports <number>: Scan number most common ports
  --port-ratio <ratio>: Scan ports more common than <ratio>
SERVICE/VERSION DETECTION:
  -sV: Probe open ports to determine service/version info
  --version-intensity <level>: Set from 0 (light) to 9 (try all probes)
  --version-light: Limit to most likely probes (intensity 2)
  --version-all: Try every single probe (intensity 9)
  --version-trace: Show detailed version scan activity (for debugging)
SCRIPT SCAN:
  -sC: equivalent to --script=default
  --script <lua scripts>: <lua scripts> is a comma separated list of
    directories, script files or script categories
  --script-args=<n1=v1,[n2=v2,...]>: provide arguments to scripts
  --script-args-file <filename>: provide NSE script args in a file
  --script-trace: show all data sent and received
  --script-updatedb: Update the script database.
  --script-help=<lua scripts>: Show help about scripts.
    <lua scripts> is a comma-separated list of script-files or
    script-categories.
OS DETECTION:
  -O: Enable OS detection
  --osscan-limit: Limit OS detection to promising targets

```

```

(kali@kali)-[~]
$ nmap --script smb-vuln 89.117.188.67
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:16 IST
Nmap scan report for 89.117.188.67
Host is up (0.028s latency).
Not shown: 962 filtered tcp ports (no-response), 34 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
80/tcp    open  http
443/tcp   open  https
3306/tcp  open  mysql

Nmap done: 1 IP address (1 host up) scanned in 4.12 seconds

```

```

(kali@kali)-[~]
$ nmap --script-smtp-vuln-cve2011-1704 -pT:25,465,587 kali
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:16 IST
Nmap scan report for kali (127.0.1.1)
Host is up (0.000048s latency).

PORT      STATE SERVICE
25/tcp    closed smtp
465/tcp   closed smtps
587/tcp   closed submission

Nmap done: 1 IP address (1 host up) scanned in 0.13 seconds

```

```

(kali@kali)-[~]
$ nmap --script mysql-enum vit.ac.in
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:17 IST
Nmap scan report for vit.ac.in (122.184.65.22)
Host is up (0.0029s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http
113/tcp   closed ident
443/tcp   open  https
8080/tcp  closed http-proxy

Nmap done: 1 IP address (1 host up) scanned in 5.08 seconds

```

Command 14: Enumeration Scripts

- **Commands:**
 - **nmap -sS -SU -p 53 --script dns-nsec-enum --script-args dns-nsec-enum.domains=vit.ac.in vit.ac.in:** Attempts to "walk" the DNSSEC records to enumerate all subdomains.
 - **nmap -sV -p 161 147.135.87.137:** Probes the SNMP service (port 161) to try to get system information.

- `nmap -sU -p 137 --script nbstat.nse 147.135.87.137`: Enumerates NetBIOS information from a Windows host.
- `nmap -p 389 --script ldap-search ... vit.ac.in`: Tries to query the LDAP directory service (like Active Directory) for user information.

```
(kali@kali) [~]
$ nmap -sU -p 389 --script dns nsd enum --script args dns nsd enum.dnsinfo=vit.ac.in vit.ac.in
Failed to resolve/decode supposed IPv4 source address "0": None or service not known
QUITTING!
```

```
(kali@kali) [~]
$ nmap -sV -p 161 147.135.87.137
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:18 IST
Nmap scan report for idelivery.artisanjazz.com (147.135.87.137)
Host is up (0.32s latency).

PORT      STATE SERVICE VERSION
161/tcp    closed snmp

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1.55 seconds
```

```
(kali@kali) [~]
$ nmap -sU -p 137 --script nbstat.nse 147.135.87.137
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:18 IST
Nmap scan report for idelivery.artisanjazz.com (147.135.87.137)
Host is up (0.29s latency).

PORT      STATE SERVICE
137/udp    closed netbios ns

Nmap done: 1 IP address (1 host up) scanned in 0.83 seconds
```

```
(kali@kali) [~]
$ nmap -p 389 --script ldap-search ... vit.ac.in
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:19 IST
Failed to resolve "...".
Nmap scan report for vit.ac.in (122.184.65.22)
Host is up (0.0029s latency).

PORT      STATE SERVICE
389/tcp    filtered ldap

Nmap done: 1 IP address (1 host up) scanned in 0.49 seconds
```

Nmap Firewall Evasion

Command 15: Fragmented Packets

- Command: `nmap -f <target IP>`

- The **-f** flag fragments the scan packets into smaller 8-byte pieces. This is a very old technique used to bypass simple firewalls or packet filters that only inspect the first fragment of a packet.

```
(kali@kali)-[~]
$ nmap -f 172.17.98.253
Starting Nmap 7.05 ( https://nmap.org ) at 2025-10-30 14:20 IST
Nmap scan report for 172.17.98.253
Host is up (0.0000050s latency).
All 1000 scanned ports on 172.17.98.253 are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 1 IP address (1 host up) scanned in 0.13 seconds
```

Command 16: Source Port Manipulation

- Command: **nmap --source-port 53 <target IP>**
- This makes the scan packets *appear* to be coming from port 53 (DNS). Some poorly configured firewalls trust traffic from "well-known" ports like 53 or 80, and will allow the scan packets in .

```
(kali@kali)-[~]
$ nmap --source-port 53 172.17.98.253
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:21 IST
Nmap scan report for 172.17.98.253
Host is up (0.0000050s latency).
All 1000 scanned ports on 172.17.98.253 are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 1 IP address (1 host up) scanned in 0.13 seconds
```

Command 17: Decoy Scanning

- Command: **nmap -D RND:10 <target IP>**
- This is a fantastic evasion technique. It launches the scan from your *real* IP, but it also spoofs 10 random decoy IPs (RND:10) that send packets at the same time. The target's logs will show the scan coming from 11 different IPs, making it much harder to identify the real attacker

```
(kali@kali)-[~]  
$ nmap -D RND:10 172.17.98.253  
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 14:21 IST  
Nmap scan report for 172.17.98.253  
Host is up (0.000033s latency).  
All 1000 scanned ports on 172.17.98.253 are in ignored states.  
Not shown: 1000 closed tcp ports (reset)  
  
Nmap done: 1 IP address (1 host up) scanned in 1.32 seconds
```

Command 18: Idle Scan

- Command: `nmap -sI <zombie IP> <target IP>`
- This is the ultimate stealth scan. It requires a third-party, "zombie" machine on the internet. Nmap sends *no packets* from your IP to the target. Instead, it "bounces" the scan off the zombie machine and measures changes in its IP ID sequence. This allows you to scan the target completely anonymously, as the target's logs will only show the zombie's IP